

OpenDeveloper Reference Manual



OpenDeveloper Reference Manual

Copyright

©2002-2006 Tucker-Davis Technologies, Inc. (TDT). All rights reserved.

No part of this manual may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording, for any purpose without the express written permission of TDT.

Licenses and Trademarks

Windows Vista and Windows XP are registered trademarks of Microsoft Corporation.

Table of Contents

BEFORE YOU BEGIN:	1
REQUIREMENTS.....	1
ORGANIZATION OF THE MANUAL.....	1
OVERVIEW	3
TTANKX	5
ABOUT THE TTANK ACTIVE X CONTROL.....	5
ORGANIZATION OF TTANKX METHODS	5
GLOBAL PARAMETER DEFAULTS	6
EXAMPLES.....	9
<i>Example: Extract Waveforms and Event Data</i>	9
<i>Example: Using Epochs to Filter Data</i>	12
<i>Example: Reconstructing Waveforms from Events</i>	15
<i>Example: Using Filter Arrays</i>	20
<i>Example: Plotting Data in an Inter-Spike Interval Histogram</i>	25
ACCESS CONTROL TTANK X	28
<i>ConnectServer</i>	28
<i>ReleaseServer</i>	28
<i>OpenTank</i>	28
<i>CloseTank</i>	29
<i>CheckTank</i>	29
<i>SelectBlock</i>	30
RETRIEVING RECORDS TTANK X	30
<i>GetSortName</i>	30

<i>ReadEvents</i>	31
<i>ReadEventsSimple</i>	33
<i>ReadEventsV</i>	33
<i>ReadWavesOnTimeRangeV</i>	34
<i>ReadWavesOnTimeRangeB</i>	35
<i>ReadWavesV</i>	36
<i>ParseEv</i>	37
<i>ParseEvInfoV</i>	38
<i>ParseEvV</i>	40
<i>SetUseSortName</i>	40
EPOCHS AND FILTERING TTANK X.....	41
<i>About Epochs and Filtering</i>	41
<i>CreateEpocIndexing</i>	42
<i>ClearEpocIndexing</i>	42
<i>GetEpocCode</i>	42
<i>GetEpocsV</i>	43
<i>GetEpocsExV</i>	44
<i>GetFilterTolerance</i>	45
<i>GetValidTimeRangesV</i>	46
<i>QryEpocAt</i>	46
<i>QryEpocAtV</i>	47
<i>ResetFilters</i>	48
<i>SetEpocTimeFilter</i>	48
<i>SetEpocTimeFilterV</i>	49
<i>SetEpocTimeFilterB</i>	50

<i>SetFilter</i>	51
<i>SetFilterArray</i>	52
<i>SetFilterWithDesc</i>	53
<i>SetFilterWithDescEx</i>	54
<i>SetFilterTolerance</i>	55
<i>SetRefEpoc</i>	55
<i>SetRefEpocV</i>	56
<i>SetRefEpocB</i>	57
INFORMATION ACCESS TTANK X	57
<i>CurBlockMemo</i>	57
<i>CurBlockName</i>	57
<i>CurBlockNotes</i>	58
<i>CurBlockOwner</i>	58
<i>CurBlockStartTime</i>	59
<i>CurBlockStopTime</i>	59
<i>FancyTime</i>	59
<i>GetCodeSpecs</i>	60
<i>GetEnumServer</i>	60
<i>GetEnumTank</i>	61
<i>GetError</i>	61
<i>GetEventCodes</i>	62
<i>GetGlobalStringV</i>	62
<i>GetGlobalStringB</i>	62
<i>GetGlobalV</i>	63
<i>GetGlobalB</i>	63

<i>GetHotBlock</i>	63
<i>GetStatus</i>	64
<i>GetTankItem</i>	65
<i>QueryBlockName</i>	65
MISC UTILITIES TTANK X	66
<i>CodeToString</i>	66
<i>DFromToString</i>	66
<i>EvTypeToString</i>	66
<i>GetNPer</i>	67
<i>ResetGlobals</i>	67
<i>SetGlobalV</i>	68
<i>SetGlobalB</i>	68
<i>SetGlobals</i>	69
<i>SetGlobalsB</i>	69
<i>SetGlobalStringV</i>	70
<i>SetGlobalStringB</i>	70
<i>StringToEvCode</i>	70
TTANKINTERFACES	73
ABOUT THE TTANKINTERFACES	73
<i>Properties, Methods, and Events</i>	73
SERVERSELECT TTANKINTERFACES	75
<i>About ServerSelect</i>	75
<i>ActiveServer, Properties</i>	75
<i>AllowEdit, Properties</i>	75
<i>AddServer, Methods</i>	75

<i>DeleteServer, Methods</i>	76
<i>Refresh, Methods</i>	76
<i>ServerChanged, Events</i>	77
TANKSELECT TTANKINTERFACES	77
<i>About TankSelect</i>	77
<i>ActiveTank, Properties</i>	77
<i>AllowPopUp, Properties</i>	78
<i>ShowServer, Properties</i>	78
<i>SingleClickSelect, Properties</i>	78
<i>HideDetails, Properties</i>	79
<i>UseServer, Properties</i>	79
<i>Refresh, Methods</i>	79
<i>TankChanged, Events</i>	80
BLOCKSELECT TTANKINTERFACES	80
<i>About BlockSelect</i>	80
<i>ActiveBlock, Properties</i>	80
<i>AllowPopUp, Properties</i>	81
<i>HideDetails, Properties</i>	81
<i>ShowDate, Properties</i>	81
<i>ShowDuration, Properties</i>	82
<i>ShowMemo, Properties</i>	82
<i>ShowOwner, Properties</i>	83
<i>ShowStart, Properties</i>	83
<i>ShowStop, Properties</i>	83
<i>ShowTank, Properties</i>	84

<i>SingleClickSelect, Properties</i>	84
<i>UseServer, Properties</i>	84
<i>UseTank, Properties</i>	85
<i>Refresh, Methods</i>	85
<i>BlockChanged, Events</i>	85
EVENTSELECT TTANKINTERFACES	86
<i>About EventSelect</i>	86
<i>ActiveEvent, Properties</i>	86
<i>AllowActive, Properties</i>	86
<i>HideDetails, Properties</i>	87
<i>ShowBlock, Properties</i>	87
<i>ShowDataFormat, Properties</i>	87
<i>ShowFirstTime, Properties</i>	88
<i>ShowSampleFreq, Properties</i>	88
<i>ShowServer, Properties</i>	89
<i>ShowSize, Properties</i>	89
<i>ShowTank, Properties</i>	89
<i>ShowType, Properties</i>	90
<i>SingleClickSelect, Properties</i>	90
<i>UseBlock, Properties</i>	90
<i>UseServer, Properties</i>	91
<i>UseTank, Properties</i>	91
<i>Refresh, Methods</i>	91
<i>GetEvName, Methods</i>	92
<i>ActEventChange, Events</i>	92

TTANKINTERFACES EXAMPLE	92
TDEVACC	97
ABOUT TDEVACC.....	97
ORGANIZATION OF TDEVACC METHODS	98
SETUP AND CONTROL TDEVACC X.....	98
<i>ConnectServer</i>	98
<i>CheckServerConnection</i>	98
<i>GetSysMode</i>	99
<i>SetSysMode</i>	99
<i>SetTankName</i>	100
<i>GetTankName</i>	100
<i>CloseConnection</i>	100
HARDWARE DATA ACCESS TDEVACC X.....	101
<i>SetTargetVal</i>	101
<i>GetTargetVal</i>	101
<i>WriteTarget</i>	102
<i>WriteTargetV</i>	102
<i>WriteTargetVEX</i>	103
<i>ReadTarget</i>	104
<i>ReadTargetV</i>	104
<i>ReadTargetVEX</i>	105
HARDWARE INFORMATION RETRIEVAL TDEVACC X.....	106
<i>GetDeviceRCO</i>	106
<i>GetDeviceSF</i>	106
<i>GetDeviceStatus</i>	106

<i>GetDeviceType</i>	108
<i>GetNextTag</i>	109
<i>GetTargetType</i>	110
<i>GetTargetSize</i>	110
EXAMPLES	111
KNOWN ANOMALIES	113
INDEX	115

Before You Begin:

Requirements

TDT Drivers and the OpenEx Suite must be installed before installing OpenDeveloper.

The recommended operating system for all TDT systems is 32-bit Windows XP®.

Organization of the Manual

This manual is organized in the following sections:

- Overview
- TTankX
- TTankInterfaces
- TDevAcc

Overview

OpenDeveloper is a group of ActiveX controls that can be used with programming languages such as MATLAB, Visual Basic, and Visual C++ to generate client applications that access the OpenEx servers (TTank and OpenWorkbench). OpenDeveloper provides access to the same set of server interfaces used by OpenEx applications such as OpenScope and OpenController.

How the TTank Server Works

The TTank server is a database engine allowing clients to access tanks and tank data. Each server supports multiple clients and multiple tanks can be open. When a server is called a connection to a local or remote server is opened and a client is added to the server. Each client can then open and access any tank on that server. Once the tank is opened the client application can filter, read, write, and even remove data from the tank. Several applications can run simultaneously on a single or multiple computers and each application can access information from the server.

How the OpenWorkbench Server Works

The OpenWorkbench server offers an interface for use by other client programs like OpenController. This interface allows client programs to access system hardware while experiments are running. This access allows client applications to acquire data for real-time analysis and to control experimental parameters dynamically. This interface also supports control of OpenWorkbench system modes (Record, Preview, Standby, and Idle) via the SetMode and GetMode calls.

Open Developer ActiveX Controls

There are three ActiveX controls that ship with OpenDeveloper: TTankX, TTankInterface, and TDevAcc. They are simply wrappers for the underlying COM interfaces offered by the TTank and OpenWorkbench servers. Providing an ActiveX wrapper allows for a standardized interface for a number of programming environments.

TTankX is a list of methods that access the TTank server and tanks on that server.

TTankInterface offers a set of GUI interfaces for tank management for use in your programs.

TDevAcc wraps the interfaces offered by OpenWorkbench. These interfaces give real-time access to system hardware.

Before Using OpenDeveloper

TDT recommends that users become familiar with how OpenEx works before they use OpenDeveloper. In particular, users should examine how OpenWorkbench, OpenController, and OpenBrowser function.

TTankX

About the TTank ActiveX Control

The TTank ActiveX control is a wrapper for the interfaces offered by TTankServer. This control also implements a number of support methods. It handles many of the details of client access to TTankEngine and is the ONLY user access supported by OpenDeveloper and TDT.

Users should be mindful of using good 'closed loop' access when working with TTankX. This means always closing your tanks and releasing your servers.

A typical tank access session for a client consists of six main steps:

1. **Call ConnectServer** -- Called to connect to TTankServer. The connection is terminated with ReleaseServer.
2. **Call OpenTank** -- Called to open access to the desired tank. Access is terminated with CloseTank.
3. **Use SelectBlock** -- Used to select the desired block within an open tank. SelectBlock does not need to be terminated.
4. Perform any number of access operations on data within a selected block.
5. **Call CloseTank** -- Called to close a tank opened with OpenTank. CloseTank can also be called when the user is unsure whether or not a tank is open.
6. **Call ReleaseServer** -- Closes the connection to a server. Will also close any open tank.

Typical MATLAB code might look something like this:

```
TT = actxcontrol('TTank.X')
TT.ConnectServer('Local','MyClient')
TT.OpenTank('MyTank','R')
TT.SelectBlock('Block-1')
%Place your application specific operations here
TT.CloseTank
TT.ReleaseServer
```

Organization of TTankX Methods

TTankX methods can be divided into five basic groups:

- **Access Control** -- The methods in this group are used to control access to a server, tank, and block of data.
- **Retrieving Records** -- The methods in this group are used to read and retrieve events and event information for use with the client application.
- **Epochs and Filtering** -- The methods in this group are used to index and filter events and epochs.
- **Information Access** -- The methods in this group are used to retrieve information such as lists of the available servers, tanks, blocks, events, and epochs; and to check the status of an open tank.

- **Misc Utilities** -- The methods in this group are used to perform common tasks such as converting code formats and determining the size of a data format.

Detailed information about each TTankX method is organized according to these five groups and provided in the pages that follow.

Global Parameter Defaults

The original core set of OpenDeveloper, TTankX methods each used its own group of arguments (such as channel or offset) to affect the functioning of the method. In all newer calls, the number of arguments for each individual method has been drastically reduced. Instead, they use global parameters which are set with default values and need not be declared unless a different value is desired. The global parameters can be changed using SetGlobals, page 69, SetGlobalV, page 68, or SetGlobalStringV, page 70.

AutoRefEpoch

Default: 1 (enabled)

Description: When using the call SetEpocTimeFilter, the time stamps of the events are referenced to the onset of that epoch event. This makes the construction of histograms easier. If this referencing of time stamps is not desired, then the AutoRefEpoc global parameter must be set to 0.

Valid values: 0 (disabled) or 1 (enabled)

Channel

Default: 0 (all channels)

Description: Specifies that all channels will be used. Set to some number to specify a channel number.

Valid values: any integer

FillItem

Default: 'DataPoints'

Description: Specifies that the returned matrix for ReadWavesV and similar methods will be filled with actual data points.

Valid values:

'Name' – Name of the event

'Channel' – Channel number of the event

'Sort' – Sort code of the event

'Time' – Time stamp of the event

'Freq' – Sampling rate of the event

'xIndex' – Index along the x dimension (used with SetFilterArray)

'yIndex' – Index along the y dimension (used with SetFilterArray)

'zIndex' – Index along the z dimension (used with SetFilterArray)

'FixedNum' – Arbitrarily specified number. The number can be specified using 'Fillvalue'

FillValue**Default:** 1**Description:** If FillItem has been specified as FixedNum, this parameter specifies an arbitrary number to be inserted into the matrix at the occurrence of each event.**Valid values:** any number**MaxReturn****Default:** 100000**Description:** Specifies the maximum number of events to be returned. Used with ReadEventsSimple, GetEpocsExV, and GetValidTimeRanges, but not ReadWaves and similar methods.**Valid values:** any integer**Options****Default:** 'ALL'**Description:** Specifies that all events are extracted, not just filtered or new.**Valid values:** 'FILTERED', 'NEW', and other values valid for the method being used.**RespectOffsetEpoc****Default:** 1**Description:** This affects only buddy epochs or those epochs that have an offset. When set to 1 it will filter out the events that occur after the offset of the buddy epoch, otherwise it will include all events until the next onset.**Valid values:** 0**SortCode****Default:** 0 (any)**Description:** Specifies the inclusion of spikes with all sort codes. **Note:** in many OpenEx applications, 0 is used for unassigned spikes. Here, 0 encompasses all sort codes.**Valid values:** any integer up to 32

If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.

T1**Default:** 0.0**Description:** When events are being extracted, this parameter specifies the starting point for that extraction.**Valid values:** any number

T2

Default: 0.0

Description: When events are being extracted, this parameter specifies the stopping point for the extraction. 0.0 is used to specify the end of the block, unless the number of events exceeds the MaxReturn.

Valid values: any number

WavesMemLimit

Default: 33554432 (~32 MB)

Description: Refers to the maximum memory that can be used by a single call. Related to ReadWavesV, ReadWavesOnTimeRangeV, which return the events in a matrix.

Valid values: any number in bytes

WaveSF

Default: Native, else 100 Hz (if there is no sampling frequency for the event. For example, an epoch event)

Description: Specifies the sampling frequency, in Hz, used to sample a certain event. Most useful when displaying multiple events with different sampling rates on the same plot.

Valid values: any number

WaveSFEvent

Default: Native

Description: Specifies an event whose sampling frequency is used to sample a certain event. Most useful when you want to display multiple events with different sampling rates on the same plot.

Valid values: any event name or event code (except epoch events) present in the block.

Examples

Example: Extract Waveforms and Event Data

This example demonstrates how data can be extracted from a tank and parsed using ReadEventsV, ParseEvV, and ParseEVInfoV. OpenDeveloper also includes other calls, such as ReadWaves and ReadWavesV, for extracting data. See Example: Reconstructing Waveforms from Events, page 15, for an example using ReadWavesV.

The example demonstrates:

- Extracting events
- Extracting time stamps

Methods used:

- [ReadEventsV](#)
- [ParseEvV](#)
- [ParseEvInfoV](#)

Note: most methods return 1 if they are successful or 0 if unsuccessful. If a variant is returned, then a -1 means Empty variant or a NULL is returned.

Example File

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\Example1.m

Or for older versions of Matlab

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\Example1_R12.m

Accessing the Tank

The first section of the Matlab script connects to the TTank ActiveX control and opens the server, tank, and block. See About the TTank ActiveX Control, page 5, for more information.

```
TTX = actxcontrol('TTank.X')
TTX.ConnectServer('Local', 'Me')
TTX.OpenTank('DemoTank', 'R')
TTX.SelectBlock('Block-3')
```

Extracting Data

The ReadEventsV method reads the event records for the specified event and caches them to local memory. Elements of the returned data can be parsed out using the ParseEvV and ParseEvInfoV calls. Here, all of the records for the Snip event are read. The arguments specify a maximum of 10,000 events to be returned. The data will include events from channel one and all sort codes. For each event, data will be included for the entire length of the event (Start = 0.0 and End = 0.0). *N* will equal the number of events read.

```
N = TTX.ReadEventsV(10000, 'Snip', 1, 0, 0.0, 0.0, 'ALL')
```

ParseEvV is used to get all waveform data for all events. Below, the first argument, 0, is the offset into the indexed list read and cached above. The second argument is the number of events to be

parsed out and returned; in this case N equals the total number of events. The data is returned in a matrix with each column containing one event

```
.
W = TTX.ParseEvV(0, N);
```

Data points for each snippet				
V	Snip 1	Snip 2	Snip 3	-->
	{.	.	.	.
	.	.	.	.
	.	.	.	.
	.	.	.	.
	.	.	.	.}

Extracting Time Stamps and Other Event Information

ParseEvInfoV can be used to return additional information about the cached events. This method uses both arguments used with ParseEvV and also requires you to specify which event information to return. The table below lists the valid values for the third argument.

Parameter	Event Information Returned
1	Amount of waveform data in bytes
2	Record Type
3	Event Code Value
4	Channel Number
5	Sorting Number
6	Time Stamp
7	Scalar Value (only valid if no data is attached)
8	Data format code
9	Data sample rate in Hz (not valid unless data is attached)
10	Not used, returns 0
0	Returns all values above

The code segment below uses ParseEvInfoV to return the time stamp values for all events. The data is returned in a matrix with a single row.

```
TS = TTX.ParseEvInfoV(0, N, 6);
```

To visualize the data, a simple subplot is generated that contains both a plot and histogram of the data.

```
subplot(2,1,1); plot(W);
subplot(2,1,2); Hist(TS, 30);
% Create a figure displaying both plots
```

The same method is used, to return a single value, in this case, the channel number. Because the offset is 35, the channel number of the 36th event will be returned.

```
chan = TTX.ParseEvInfoV(35, 0, 4)
```

More Options for Extracting Data

To return a single waveform, ParseEvV is used. The data points for the waveform are returned in a matrix with a single row.

```
onewave = TTX.ParseEvV(35, 0)
```

ParseEvV can also be used to return the value(s) for an entire group of scalar events. The data is returned in a matrix with a single row.

```
N = TTX.ReadEventsV(10000, 'Freq', 0, 0, 0.0, 0.0, 'ALL')
freqlist = TTX.ParseEvV(0, N);
```

Finally, a single scalar value (the 4th Freq value) is returned.

```
onefreq = TTX.ParseEvV(3, 0)
```

Closing the Tank

When all tasks are complete, the tank should be closed to release the connection to the server.

```
TTX.CloseTank
TTX.ReleaseServer
```

Example: Using Epochs to Filter Data

This example demonstrates how to extract filtered data from the tank. Data is filtered based on epoch events; then events are extracted and parsed for later analysis and display.

The example demonstrates:

- Extracting information about epochs
- Using Filters

Methods used:

- [CreateEpocIndexing](#)
- [QryEpocAtV](#)
- [ReadEventsV](#)
- [ParseEvV](#)
- [ParseEvInfoV](#)
- [SetFilterWithDesc](#)
- [ResetFilters](#)

Example File

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\Example2.m

Or for older versions of Matlab

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\Example2_R12.m

Accessing the Tank

The first section of the Matlab script connects to the TTank ActiveX control and opens the server, tank, and block. See About the TTank ActiveX Control, page 5, for more information.

```
TTX = actxcontrol('TTank.X')
TTX.ConnectServer('Local', 'Me')
TTX.OpenTank('DemoTank', 'R')
TTX.SelectBlock('Block-3')
```

Building an Epoch Index

The CreateEpocIndexing method is used to build epoch indexes. Indexing allows data to be accessed relative to epochs.

```
TTX.CreateEpocIndexing
```

Note: A tilde appended to the block name (such as `MyBlock = '~Block-2'`), can also serve as a shortcut to call the CreateEpocIndexing method. See Plotting Data in an Inter-Spike Interval Histogram, page 25, for an example.

Getting Epoch Information Relative to an Event

Because the index has been created, you can quickly get epoch information relative to some event data. The demo tank data includes three epochs: SwpN, Freq, and Levl. The code segment below reads all of the event data for channel one, and then queries to find out what the stimulus frequency (Freq) was when a particular event occurred.

```
N = TTX.ReadEventsV(10000, 'Snip', 1, 0, 0.0, 0.0, 'ALL')
% Read the Snip event data for channel one
TS = TTX.ParseEvInfoV(0, N, 6);
% Parse the time stamp for all events
F = TTX.QryEpocAtV('Freq', TS(13), 0)
% Query for the epoch value for Freq during the 13th event
```

QryEpocAtV returns the value associated with a specified epoch. One of four values is returned depending on the last argument: the value of the epoch (0), the start of the epoch event (1), the end of the epoch event (2), or the filter status (3). CreateEpocIndexing must be called before QryEpocAt.

Using Epoch as Filters

After the epoch index is created, you can also issue filter commands before reading data. First, use ResetFilters to ensure that you do not filter data that has already been filtered.

```
TTX.ResetFilters
```

Next, a filter is issued (SetFilterWithDesc) to get only event data for the Freq=4000.

```
TTX.SetFilterWithDesc('Freq=4000')
```

Now a call to ReadEventsV returns only events that occur when Freq is 4000. Note that the last argument has been changed from ALL to FILTERED.

```
N = TTX.ReadEventsV(10000, 'Snip', 1, 0, 0.0, 0.0, 'FILTERED')
```

SetFilter can be called multiple times to create ORed logic for the same epoch name. For example, when the statement below is added, the ORed logic is applied to yield a new filter and any events at Freq=1000 OR 4000 will be returned.

```
TTX.SetFilterWithDesc('Freq=1000')
N = TTX.ReadEventsV(10000, 'Snip', 1, 0, 0.0, 0.0, 'FILTERED')
```

ReadEventsV now returns N as 46.

Filters set for different epochs are ANDed. When the statement below is added, the ANDed logic is applied and only events that have Freq=1000 or Freq=4000 and SwpN=13 will be returned.

```
TTX.SetFilterWithDesc('SwpN=13')
```

```
N = TTX.ReadEventsV(10000, 'Snip', 1, 0, 0.0, 0.0, 'FILTERED')
```

N returns 0, meaning that no events fit the criteria.

Closing the Tank

When all tasks are complete, the tank should be closed to release the connection to the server.

```
TTX.CloseTank  
TTX.ReleaseServer
```


Example: Reconstructing Waveforms from Events

This example demonstrates the steps used to reconstruct waveforms from events. It describes each step typically used after an experiment. First, data is filtered based on epoch events. Next, the filtered data is extracted from the tank and waveforms are built from the events.

The example demonstrates:

- Using Global parameters.
- Filtering signal data based on epoch events.
- Matching up data sets with different sampling rates for later display.

Methods used:

- [SetGlobalV](#)
- [SetGlobalStringV](#)
- [ResetFilters](#)
- [SetFilterWithDescEx](#)
- [ReadWavesV](#)
- [GetValidTimeRangesV](#)
- [ReadWaveOnTimeRangeV](#)

Note: Because this example uses ReadWavesV, which extracts the data in a format that simplifies plotting but requires a great deal of memory, the program takes a few seconds to be executed.

Example File

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\WaveReconstruction.m

Or for older versions of Matlab

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\WaveReconstruction_R12.m

Accessing the Tank

The first section of the Matlab script connects to the TTank ActiveX control and opens the server, tank, and block. See About the TTank ActiveX Control, page 5, for more information.

```
MyTank = 'DemoTank';
MyBlock = '~Block-2';
TTX = actxcontrol('TTank.X')
TTX.ConnectServer('Local','Me')
TTX.OpenTank(MyTank,'R')
TTX.SelectBlock(MyBlock)
```

Building an Epoch Index

In this example, when the block is defined, a tilde is appended to the block name (such as `MyBlock = '~Block-2'`), serving as a shortcut to call the CreateEpochIndexing method. This method is used to build epoch indexes which allow data to be accessed relative to epochs.

Using Global Parameters

Global parameters reduce the argument list for each method. TDT sets up default settings for these method calls (see global parameters for more information, page 6). To modify the global settings, a method call of either SetGlobalV or SetGlobalStringV is generated (depending on the global variable). The method call sets a Global parameter (in this case, the parameter 'Channel' is set to 0 meaning all channels). To set the global parameter for using filtered data, the SetGlobalStringV method sets the parameter 'Options' to FILTERED.

```
TTX.SetGlobalV('Channel',0);
TTX.SetGlobalStringV('Options','FILTERED');
```

Resetting filters ensures that you do not filter a subset of your data.

```
TTX.ResetFilters;
```

Filtering and Processing Data

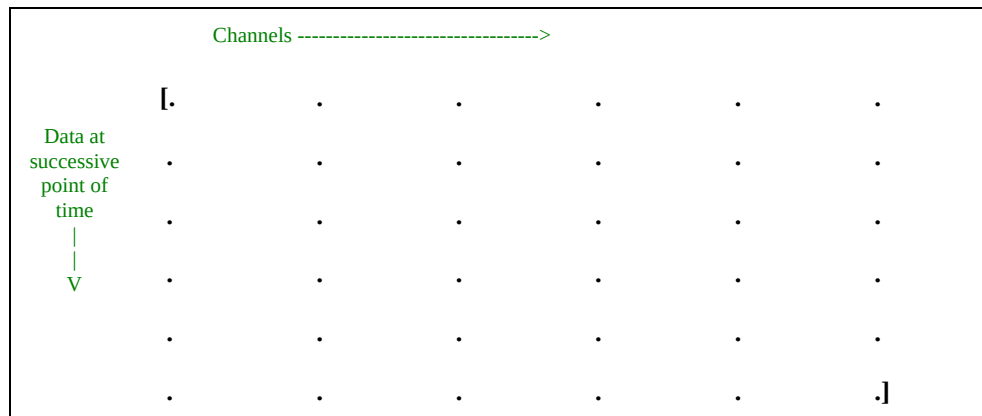
All filters are based on epochs. Epochs are scalar variables that are associated with fixed events, such as a behavioral response or stimulus presentation. In this example, the epochs are information about auditory stimuli, such as frequency and level. The next line sets the data filters. This command sets the data filters so that only data that occurred during epochs that had a Freq of 2000 and a Lev1 of 0 is read.

```
TTX.SetFilterWithDescEx('Freq=2000 and Lev1=0')
```

ReadWavesV reads back the snippet data and forms a waveform from the points obtained for each channel.

```
filtSpikes = TTX.ReadWavesV('Snip');
```

ReadWavesV returns the events in a matrix. The diagram below illustrates the structure of the data matrix, with each column containing the waveform data for a channel and rows listing points in time.



In many cases, you'll want to view multiple events along the same axis. The next section of the code reads data for the Freq epoch so that it can be plotted with the snippet data already extracted. Before extracting epoch events, you'll need to consider two issues.

First, consider the sampling rates of the events of interest. For scalar data, the data is sampled in an asynchronous fashion, so there is no native sample rate. Unless specified, epoch events are extracted using a fixed sampling rate of 100 Hz. In this case, we need to generate a matrix that interleaves the Epoch event with zeros so that the Y dimension of the snippet and event data match.

Second, consider the difference in magnitude between the events. In many cases, the events will differ by orders of magnitude. Viewing the data on the same axis requires conversion of one value into another. In some cases, it may require scaling the values in the matrix. In other cases it may require replacing one value by another. In this case, we already know the value of the epoch (it is the epoch we filtered on) so we replace the value with one of the same magnitude.

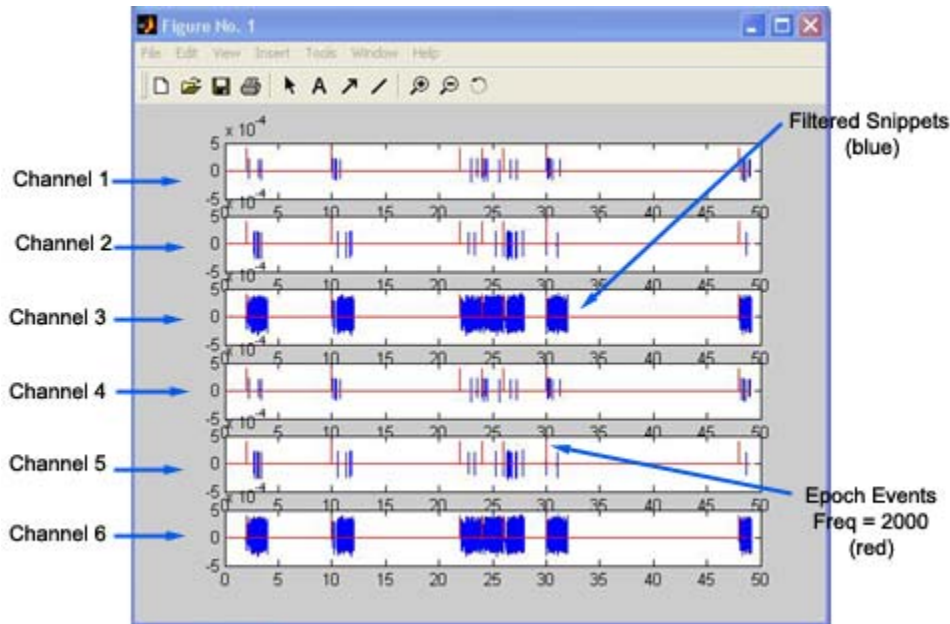
Both of these issues can be resolved using some global variables. The global variable FillItem replaces each data point of the epoch with a fixed value. This fixed value must be specified by setting the global parameter FillValue. The global variable WaveSF sets the sampling frequency to 24414 (the sampling frequency of the Snip event) so that both data sets can be plotted on the same X-axis.

```
TTX.SetGlobals('FillItem=FixedNum;FillValue=0.0004');
TTX.SetGlobalV('WaveSF',24414)
```

After the global variables are set, ReadWavesV is used to read the epoch data.

```
filtFreqs = TTX.ReadWavesV('Freq');
```

After both data sets have been read and returned as Matlab matrices, standard Matlab scripting is used to set up an array for the time axis and to plot each of the six channels of snippet data (Snip) in a subplot. Superimposed upon that plot, is a plot of the occurrences of the frequency epoch (Freq) in red.



A second way to view the data is to compare filtered data from the same channel across epoch events. The next section of the script generates a second plot to display data for two channels, with a subplot for each event.

Keep in mind that the filters and parameters have not been changed since the last ReadWaves call. The data will once again be read after the same filter of 'Freq = 2000 and Lev1 = 0' is applied.

One global parameter that must be reset is the FillItem parameter. Earlier, this parameter was set to FixedNum. To ensure that actual values are acquired the next time data is read, it must now be set to DataPoints

```
TTX.SetGlobals('FillItem=DataPoints');
```

Next, the valid duration of each event (GetValidTimeRangesV) must be established and used to read back the snippet events for only those valid time ranges. In this example, the task is broken into three steps. First, GetValidTimeRangesV displays the time range values in the command window. Second, GetEpochsExV identifies the individual epochs to be read. Finally, ReadWavesOnTimeRangeV reads the data for each valid time range or epoch.

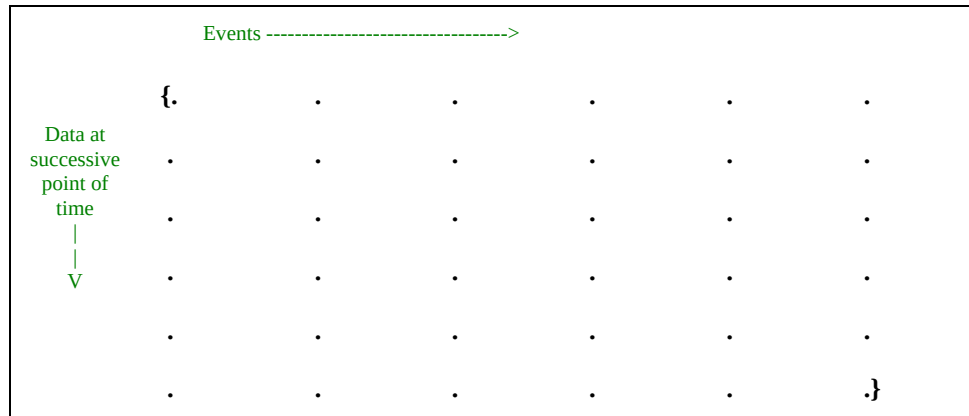
```
Ranges = TTX.GetValidTimeRangesV
TimeRanges = TTX.GetEpochsExV('Freq',0)
Chan1 = TTX.ReadWavesOnTimeRangeV('Snip',1);
Chan2 = TTX.ReadWavesOnTimeRangeV('Snip',2);
```

Note that GetValidTimeRangesV and GetEpochsExV are not required for the use of ReadWavesOnTimeRangeV. However, if GetEpochsExV is not used, epochs occurring in succession (without any gap) would be identified as a single time range and would be plotted in a single subplot.

Note: when viewing the output data in the Matlab command window, the user will notice that there are three time ranges that occur in succession (22-24 sec, 24-26 sec and 26-28 sec).

The data is read into two separate matrices. Each matrix contains the response from one channel. Each column of the matrix contains a different epoch event and the rows contain the data points from the waveform for that channel. Note that, to plot the data correctly, times where the events

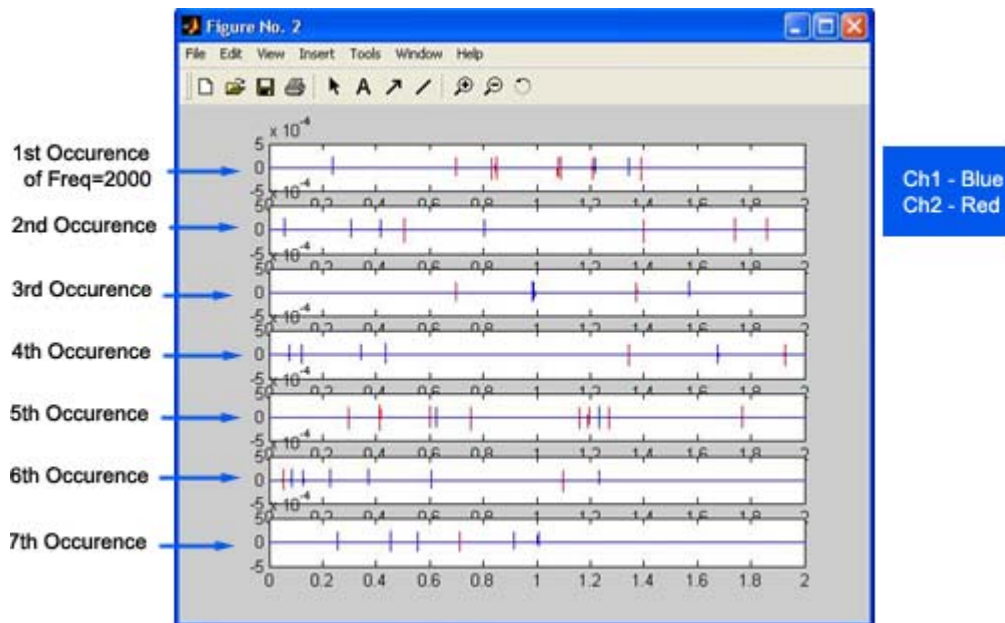
did not occur are filled with zeros. Times where the events did occur contain the neural response of the unit. The diagram below illustrates the structure of one of the data matrices.



Channel X Filtered On Epoch Event X

This structure facilitates further analysis or plotting. For example, if you wanted to determine the average response from several stimulus presentations of the frequency and intensity, it would be simple to sum across the matrix and view the aggregate response.

In this example, standard Matlab scripting is used to plot the data. Each column of the channel one matrix (the first of seven valid time ranges during which Freq=2000) is displayed in a subplot with channel two data from the same time range superimposed in a different color on the same subplot.



Finally, the server is closed.

`TTX.CloseTank`

`TTX.ReleaseServer`

Example: Using Filter Arrays

This example describes how to filter data from the OpenEx Tank. Data is filtered by querying the data tank using special events called Epochs. In OpenDeveloper, we use a three-step process to filter and load the data to local memory. The first part of the process is to set the filters. In this example, we use the ActiveX method call `SetFilterArray` that allows users to build a matrix of filter settings. Events are filtered into cells of a matrix, with each cell specifying a certain set of filters. The second part of the process is to read back the events and event information, which is done with `ReadEventsSimple`. The final part is to parse the event information for later analysis and display.

The example demonstrates:

- Filtering signal data based on epoch events.
- Using Global settings
- Constructing filter arrays using multiple `SetFilterArray` calls.

Methods used:

- [SetFilterArray](#)
- [ReadEventsSimple](#)
- [ParseEvInfoV](#)
- [ParseEvV](#)
- [SetGlobals](#)
- [SetFilterWithDescEx](#)

Example File

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\FILTERArray.m

Or for older versions of Matlab

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\FILTERArray_R12.m

Accessing the Tank

The first section of the Matlab script connects to the TTank ActiveX control and opens the server, tank, and block. See the TTankX example, page 5, for more information.

```
MyTank = 'DemoTank';
MyBlock = '~Block-2';
TTX = actxcontrol('TTank.X')
TTX.ConnectServer('Local','Me')
TTX.OpenTank(MyTank)
TTX.SelectBlock(MyBlock)
```

Building an Epoch Index

In this example, when the block is defined, a tilde is appended to the block name (such as `MyBlock = '~Block-2'`), serving as a shortcut to call the [CreateEpochIndexing](#) method. This method is used to build epoch indexes which allow data to be accessed relative to epochs.

Filtering and Processing Data

All filters are based on epochs. Epochs are scalar variables that are associated with fixed events, such as a behavioral response or stimulus presentation. In this example the epochs are information about auditory stimuli, such as frequency and level. The `SetFilterArray` command is used to set a filter and to give it an ID along a dimension. Three dimensions (0, 1, and 2) are allowed in all. Note that in other methods the dimension parameters will be defined as X, Y, and Z. Each filter has an ID along at least one dimension. Later the IDs will be used to plot the events in a grid, with each cell of the grid representing the conditions set by a filter.

The first filter is set with the condition `Freq=1000` and is given ID 1 along the 0th dimension (x-Dimension). The first parameter in the argument specifies the dimension, the second specifies the ID of the filter along that dimension, followed by the filter itself. OpenEx allows users to query the data tank through an API using common SQL language with Boolean operations such as 'and' and 'or'. The last parameter sets a flag for exclusivity of the filter. Events that fit criteria of multiple filter settings can be assigned either to the lowest ID number (exclusivity) or to each filter for which it meets the criteria. Users should consider whether setting this flag will bias their analysis.

```
a = TTX.SetFilterArray(0,1,'Freq=1000',0)
b = TTX.SetFilterArray(0,2,'Freq=2000',0)
c = TTX.SetFilterArray(0,3,'Freq=4000',0)
d = TTX.SetFilterArray(0,4,'Freq=8000',0)
e = TTX.SetFilterArray(1,1,'Levl=0',0)
```

	Dimension 0 (X) ----->				
Dimension 1(Y) v		ID1 Freq=1000	ID2 Freq=2000	ID3 Freq=4000	ID4 Freq=8000
	ID1 Levl=0	Freq=1000 and Levl=0	Freq=2000 and Levl=0	Freq=4000 and Levl=0	Freq=8000 and Levl=0

Reading Data

Next, the filtered events in 'Snip' are read from channel one. Here we use the `ReadEventsSimple` call, which reads events from the tank into local memory. Previous users of OpenDeveloper, note that `ReadEventsSimple` is a simplified version of the `ReadEvents` call. It has the same functionality as `ReadEventsV`, but uses global parameters instead of arguments.

Using Global Parameters

Global parameters were included in release 1.54 of OpenDeveloper to minimize the number of variables in each method call. This minimizes errors in typing and allows users to set parameters only once for several calls. To make OpenDeveloper backward compatible, the method calls that use global parameters are defined differently. Global parameters can be set at any point in the program, and the new value will apply to any subsequent method that uses them. This allows users to set the value of a parameter across multiple methods and eliminates the need to set parameters in each method's argument list.

The `ReadEventsSimple` method uses several global parameters whose default values are not changed in the example, such as `SortCode` (default 0, meaning all), `T1` and `T2` (both default 0, meaning full time span). Three globals, however, are changed – `MaxReturn`, `Channel` and `Options`. `MaxReturn` and `Channel` refer to the maximum number of events returned and channels extracted.

By default, the global parameter Options is set to ALL, specifying that all events are extracted, not just filtered or new events. In this case, SetGlobalV is used to set the Options to FILTERED so that only events that meet the filter criteria will be read.

```
TTX.SetGlobals('Channel=1; MaxReturn=10000; Options=FILTERED');
```

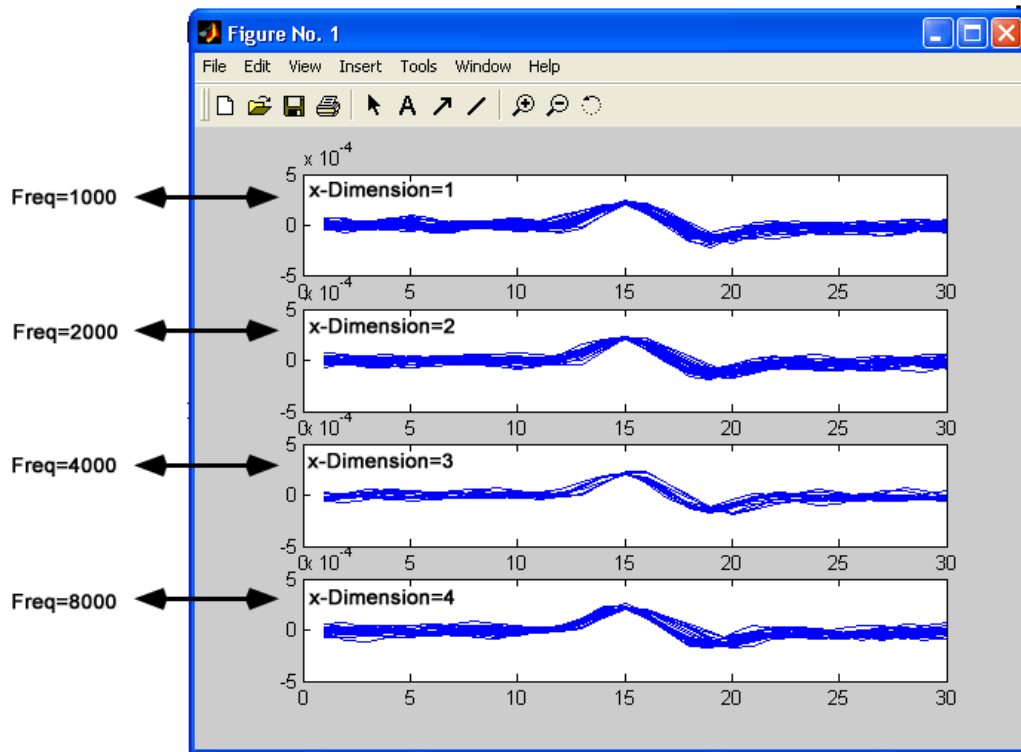
If users need to have more control over the parameters then they should use ReadEvents. ReadEventsSimple returns the number of events read. So it will return a maximum of 10000 filtered events from channel one.

```
X = TTX.ReadEventsSimple('Snip');
```

Next, the program loops through each event that was extracted from channel one of 'Snip'. Then the ID of each event along the 0th dimension is extracted using the ParseEvInfoV call. Finally, each event is extracted.

Within this loop, a grid of plots with one column and four rows (dimensions of the filter array) is formed. Then each event is plotted in that cell of the grid, which denotes its ID. For example, if an event has ID 2 along the 0th dimension, then it will be plotted in subplot two. At the end of the loop, subplot two will have only those events which have 0th dimension ID equal to 2, that is, those events which satisfy the Freq=2000 filter.

```
for t = 1:double(x)
xid  = TTX.ParseEvInfoV(t-1,1,11);
data = TTX.ParseEvV(t-1,0);
.
.
.
end
```

For the next plot, the global parameters are set again, this time with channel being 0 or All, so that data from all the channels will be viewed.

```
TTX.SetGlobals('Channel=0; MaxReturn=10000; Options=FILTERED');
```

Next the filter is set to Freq=2000, so that only events that occur when the value of the epoch 'Freq' is 2000 are extracted.

```
TTX.SetFilterWithDescEx('Freq=2000')
```

Again a filter array is created. However, this time events are differentiated by the channels on which they occurred. Note that each of these will be 'ANDed' with the previous filter, Freq=2000.

```
a = TTX.SetFilterArray(0,1,'Chan=1',0)
b = TTX.SetFilterArray(0,2,'Chan=2',0)
c = TTX.SetFilterArray(0,3,'Chan=3',0)
```

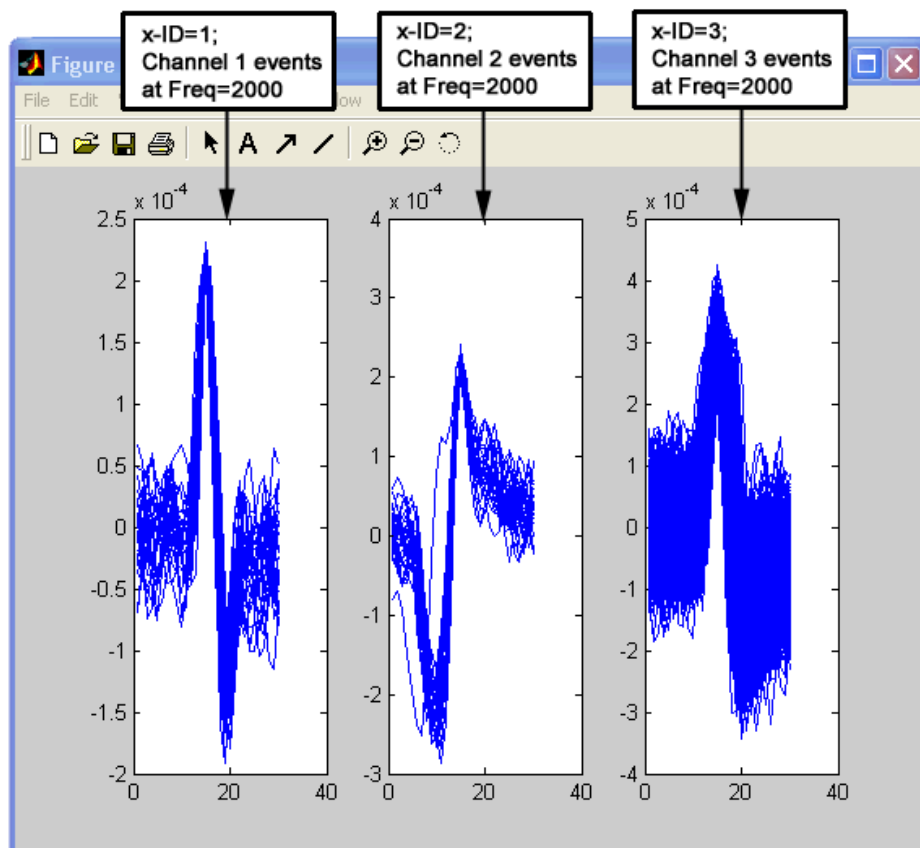
After setting up the new filters, the filtered events in 'Snip' from all channels are read again.

```
x = TTX.ReadEventsSimple('Snip');
```

Once more the program will loop through each event obtained, extract the x-dimension ID, extract the waveform itself, and plot the waveform in a subplot based on its ID. At the end of the loop,

each subplot will have only those events which have X-dimension ID corresponding to that subplot number.

```
for t = 1:double(x)
    xid = TTX.ParseEvInfoV(t-1,1,11);
    data = TTX.ParseEvV(t-1,0);
    .
    .
    .
end
```



Closing the Tank

Finally, the tank is closed and the server connection is released.

```
TTX.CloseTank;
TTX.ReleaseServer;
```

Example: Plotting Data in an Inter-Spike Interval Histogram

This example demonstrates how to access tank data and parse events. Once the data has been read, events are used to plot the inter-spike intervals (ISI). The ISI histogram is a very commonly used plot in neurophysiology analysis for determining the number of distinct firing patterns (or neurons) that have been recorded.

The example demonstrates:

- Reading time stamps for events.
- Using global variables.
- Using time stamp information to calculate and plot an inter-spike interval histogram.

Methods used:

- [SetGlobalV](#)
- [SetGlobalStringV](#)
- [ReadEventsSimple](#)

Example File

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\InterSpikeInterval.m

Or for older versions of Matlab

C:\TDT\OpenEx\Examples1.1\TTankX_Example\Matlab\InterSpikeInterval_R12.m

Accessing the Data Tank

The first section of the Matlab script connects to the TTank ActiveX control and opens the server, tank, and block. See About the TTank ActiveX Control, page 5, for more information.

```
MyTank = 'DemoTank';
MyBlock = '~Block-2';
TTX = actxcontrol('TTank.X')
TTX.ConnectServer('Local','Me')
TTX.OpenTank(MyTank,'R')
TTX.SelectBlock(MyBlock)
TTX.ResetFilters; % Reset all filters
```

Using Global Parameters and Processing Data

The ReadEventsSimple method in this example uses global parameters. ReadEventsSimple will use the default values for the global parameters unless they are specified using SetGlobalV or SetGlobalStringV (depending on the global variable). In this case, SetGlobalV is used to set the global variable Channel to include only data from channel 1.

Note: Global parameters reduce the argument list that must be specified for methods that use them. Unless specified, the default settings for these method calls are used (see global parameters for more information, page 6).

```
TTX.SetGlobalV('Channel',1);
```

A simplified version of the ReadEvents method is used to read the Snip event from channel one. ReadEventsSimple has the same functionality as the older call ReadEventsV, except that it uses global variables instead of local arguments. The number of events read will be returned. The data itself and other information will now be available for parsing in local memory.

```
a = double(TTX.ReadEventsSimple('Snip'));
```

ParseEvInfoV is used to return the time stamp values for 10000 events. The first argument specifies the data number offset, the second specifies the number of events to parse, and the last one indicates what information to parse about the event. In this case, the number 6 specifies the time stamps. The data is returned in a matrix with a single row.

```
tstamps = TTX.ParseEvInfoV(0,10000,6);
```

Creating An Inter-Spike Interval Histogram

The next section of code uses standard Matlab techniques to plot the data in an inter-spike interval histogram.

First, an array with 3500 zeroes is built to hold the 3500 bins of the histogram.

```
cache1 = zeros(1,3500);
```

Next, a loop is generated from 1 to the number of events obtained above.

```
for i = 1:a-1
```

Within the loop, the difference between each successive spike is computed.

```
delta(i) = tstamps(i+1)-tstamps(i);
```

Each value is multiplied by 1000 and rounded off so that all values are in milliseconds and the bin width is 1 ms.

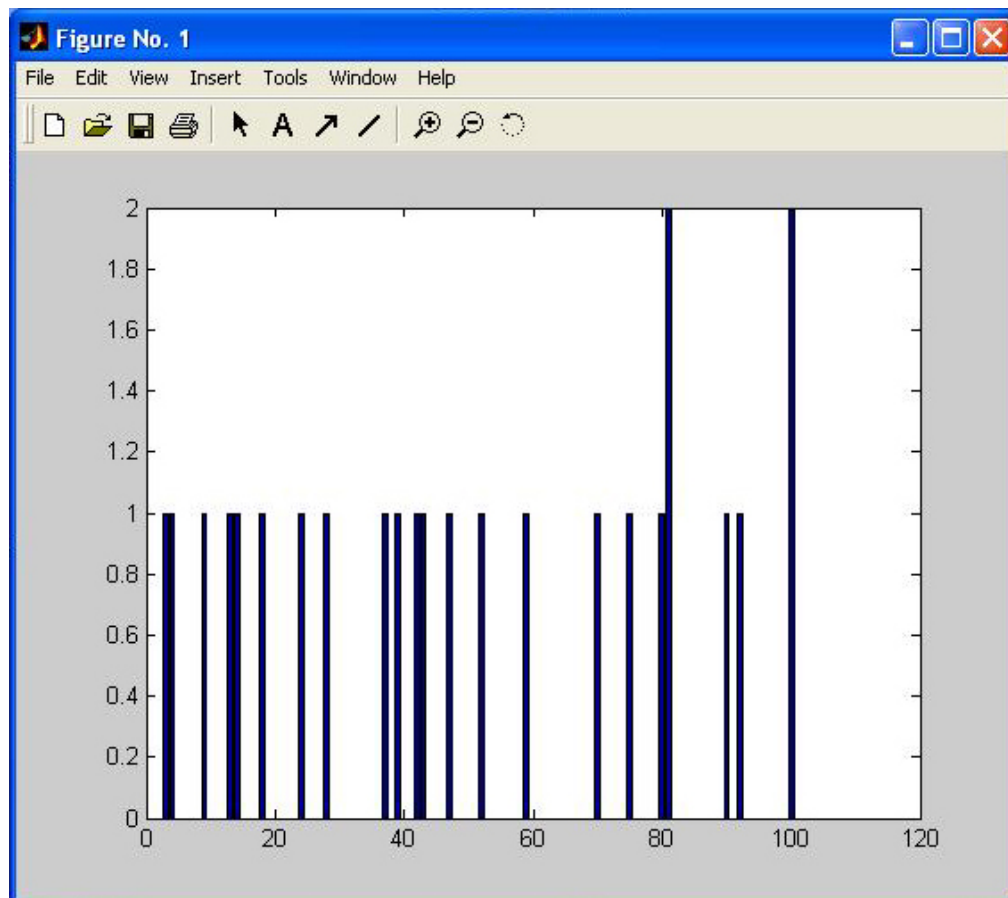
```
bin = ceil(delta(i)*1000);
```

Next, the values must be sorted into the bins. To do this, increment the value of that element of cache1, which the time stamp falls into. For example, if the time stamp extracted is 0.0399 sec (39.9 ms), then bin = 40, and we increment the 40th element of cache1. So, the end result was that an event occurred at approximately 40 ms. Since our bin width is 1 ms, that event should fall into the 40th bin, and hence the 40th element of cache1 was incremented by 1.

```
cache1(1,bin) = cache1(1,bin) + 1;
end
```

Next, the first 100 values of cache are plotted in a histogram yielding a plot of events with and inter-stimulus interval less than 100 ms.

```
bar(cache1(1:100));
```



The figure above shows an inter-stimulus interval plot generated using this example.

Closing the Tank

When all tasks are complete, the tank should be closed and the connection to the server should be released.

```
TTX.CloseTank;  
TTX.ReleaseServer;
```

Access Control

TTank X

ConnectServer

Description: ConnectServer initiates a connection with a tank server. The connection adds a client to the server. Before exiting an application the program should release the connection by calling the ReleaseServer subroutine.

Prototype: `Function ConnectServer(ServerName As String,
ClientName as String) As Long`

Arguments:

String	<i>ServerName</i>	name of the server, 'Local' is the most common
String	<i>ClientName</i>	name of the client application added to the server

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

Sample Code

Description: This code sample connects to the Local server. The Client name is 'Me'.

MATLAB `TT = actxcontrol('TTank.X')
TT.ConnectServer('Local','Me')`

Related Calls: [ReleaseServer](#)

ReleaseServer

Description: ReleaseServer releases any connected server. This method should be called when the client is finished with the server, otherwise the server application will run until the client application is closed.

Prototype: `Function ReleaseServer()`

Sample Code

Description: This code sample releases the server.

MATLAB `TT.ReleaseServer`

Related Calls: [ConnectServer](#)

OpenTank

Description: OpenTank opens a tank on the connected server for access of the specified type. The typical mode is 'R' for reading. ConnectServer must be called before OpenTank can be called. At the end of the client application use CloseTank to close the tank. To open a registered tank, use the tank name for the argument TankName. To open an unregistered tank, use the entire path to the tank.

Prototype: `Function OpenTank(TankName As String, AccessMode As String) As Long`

Arguments:

String	<i>TankName</i>	name of the tank to open			
String	<i>AccessMode</i>	'R' (most common)	'W'	'C'	'M'
		read	write	control	monitor

Returns:

Long fails (0), succeeds (1)

Sample Code

Description: This code sample connects to the local server with the client name 'Me' and opens the registered tank named MyTank for reading.

MATLAB `TT.ConnectServer('Local','Me')`
`TT.OpenTank('MyTank','R')`

Related Calls: [CloseTank](#), [ConnectServer](#)

CloseTank

Description: CloseTank closes the open tank for a client.

Prototype: `Function CloseTank()`

Sample Code

Description: This code sample opens an enumerated tank for reading. Does some operations and then closes the enumerated tank.

MATLAB `TT.OpenTank('MyTank','R')`
`% Some Code`
`TT.CloseTank`

Related Calls: [OpenTank](#), [ReleaseServer](#)

CheckTank

Description: CheckTank checks the current status of the tank.

Prototype: `Function CheckTank(TankName As String) As Long`

Arguments:

String	<i>TankName</i>	name of the tank
--------	-----------------	------------------

Returns:

Long	0	tank closed
	79	tank open
	82	tank in record mode

Sample Code

Description: This code sample connects to the local server with the client name 'Me'. Opens a tank and checks its status.

MATLAB `TT.ConnectServer('Local','Me')`
`TT.OpenTank('MyTank','R')`
`status = TT.CheckTank('MyTank')`

Related Calls: [OpenTank](#), [CloseTank](#), [GetStatus](#), [GetError](#)

SelectBlock

Description: SelectBlock selects a block from the open tank for accessing. Before this is called ConnectServer and OpenTank must be called. Each time SelectBlock is called information about the block is cached into memory.

Prototype: `Function SelectBlock(BlockName As String) As Long`

Arguments:

String	<i>BlockName</i>	name of the block
--------	------------------	-------------------

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

Sample Code

Description: This code sample connects to the server, opens a tank, and selects Block-1 from the opened tank.

MATLAB `TT.ConnectServer('Local','Me')`
`TT.OpenTank('MyTank','R')`
`TT.SelectBlock('Block-1')`

Related Calls: [OpenTank](#), [GetHotBlock](#), [ConnectServer](#)

Retrieving Records

TTank X

GetSortName

Description: GetSortName retrieves the sort IDs present for the given event name in the currently selected block. The sort IDs are returned in alphabetical order as the input argument idxSortID is incremented. If no sort IDs are present, an empty string is returned at index zero.

Prototype: `Function GetSortName(eventName As String, idxSortID As Long) As String`

Arguments:

String	<i>eventName</i>	event name
Long	<i>idxSortID</i>	sort ID

Returns:

String sortID for the given index and event name

Sample Code

Description: This code sample returns all of the sort IDs present for the Snip event in the currently selected block. The index is incremented until GetSortName returns an empty string.

```
MATLAB        idx = 0;
               sortid = 'temp';
               while ~isempty(sortid)
                 sortid = TT.GetSortName('Snip',idx)
                 idx = idx+1;
               end
```

```
VB            Dim idx As Integer
               idx = 0
               Dim sortid As String
               Do
                 sortid = TT.GetSortName("Snip", idx)
                 idx = idx + 1
               Loop Until sortid = ""
```

ReadEvents

Description: ReadEvents reads the event records for the specified EventCode from the currently selected block in the currently open tank. The events are cached to local memory within TTank.X where they can be accessed using ParseEv. Additional arguments allow you to limit the access to a particular channel and/or a particular sort code. The access can be further limited to a specified time range within the block. The Options argument allows the user to select optional access modes.

Prototype: `Function ReadEvents(MaxRet As Long, TankCode As Long, Channel As Long, SortCode As Long, T1 As Double, T2 As Double, Options As Long) As Long`

Arguments:

Long	<i>MaxRet</i>	maximum number of events to be returned Note: if the maximum number is returned it usually indicates that there were more events to be read.
Long	<i>TankCode</i>	name of event in long format
Long	<i>Channel</i>	return only records for this channel, or 0 for all channels
Long	<i>SortCode</i>	return only records with this sort code, or 0 to disregard sort codes. If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.
Double	<i>T1</i>	return all events with a time stamp greater than or equal to T1

		Note: use T1 = 0.0 to return events from the start of the block.
Double	T2	return all events with a time stamp less than T2 Note: specify T2 = 0.0 to return events to the end of the block.
Long	Options	one or more read options Note: typically this parameter is passed as 0 or GET_ALL.

Defined Name	Hex #	Function
GET_ALL	0x0000	read all event records
GET_NEW	0x0001	continue to read from last position Note: use this option to poll-read a block that is open for recording.
GET_SAME	0x0002	limit the read to the same access bounds as the previous read Note: use this option to read from a block that is open for recording.
GET_JUSTTIMES	0x0100	return a list of floats containing only the specified event time stamps
GET_DOUBLES	0x0200	same as above but return event time stamps as a list of doubles
GET_NODATA	0x0400	cause any attached waveform data NOT to be returned
GET_FILTERED	0x1000	filter returned records based on the specified filters Note: see the section on epochs and filtering.
GET_ORDERED	0x2000	not yet implemented
		All values are bit masks so that options can be used together. For example, GET_JUSTTIMES+GET DOUBLES returns just the specified event time stamps in double format.

Returns:

Long number of events cached to TTankX local buffer

Related Calls: [ReadEventsSimple](#)

ReadEventsSimple

Description: This is a highly simplified version of the call ReadEventsV. The only difference in the two calls is in the arguments that the user needs to specify each time the call is made. ReadEventsSimple uses global parameters which can be set using the SetGlobalV, SetGlobalStringV, and SetGlobals calls. See ReadEventsV, page 33, for more details.

Prototype: `Function ReadEventsSimple (EventName As String) As Long`

Arguments:

String	<i>EventName</i>	four letter event name
--------	------------------	------------------------

Also uses the following Global parameters: MaxReturn, Channel, SortCode, T1, T2, Options

If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.

Returns:

Long	number of events read
------	-----------------------

Sample Code

Description: This code sample will return the number of events read. To extract the events out of local memory, use ParseEvV and ParseEvInfoV.

MATLAB `TT.SetGlobals('Channel=1; Options=FILTERED; T2= 10')
events = TT.ReadEventsSimple('Snip')`

Related Calls: [ParseEvV](#), [ParseEvInfoV](#), [ReadWavesV](#), [ReadWavesOnTimeRangeV](#)

ReadEventsV

Description: Same as ReadEvents except the EventCode and Options parameters are specified as strings. See the ReadEvents, page 31, call for more information.

Prototype: `Function ReadEventsV(MaxRet As Long, TankCode As String, Channel As Long, SortCode As Long, T1 As Double, T2 As Double, Options As String) As Long`

Arguments:

Long	<i>MaxRet</i>	maximum number of events to be returned Note: if the maximum number is returned it usually indicates that there were more events to be read.
String	<i>TankCode</i>	name of event in 4 character string form
Long	<i>Channel</i>	return only records for this channel, or 0 for all channels
Long	<i>SortCode</i>	return only records with this sort code, or 0 to disregard sort codes. If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.
Double	<i>T1</i>	return events with time stamp greater than or equal to T1

		Note: use T1 = 0.0 to return events from the start of the block.																		
Double	T2	return events with a time stamp less than T2 Note: specify T2 = 0.0 to return events to the end of the block.																		
String	Options	The Options parameter is similar to the Options parameter for ReadEvents except that options here are specified using a list of comma separated strings. <table><tr><th>Defined Name</th><th>Same As ReadEvents Option Code...</th></tr><tr><td>"ALL"</td><td>GET_ALL</td></tr><tr><td>"NEW"</td><td>GET_NEW</td></tr><tr><td>"SAME"</td><td>GET_SAME</td></tr><tr><td>"JUSTTIMES"</td><td>GET_JUSTTIMES</td></tr><tr><td>"DOUBLES"</td><td>GET_DOUBLES</td></tr><tr><td>"NODATA"</td><td>GET_NODATA</td></tr><tr><td>"FILTERED"</td><td>GET_FILTERED</td></tr><tr><td>"ORDERED"</td><td>GET_ORDERED</td></tr></table> Options can be combined in a comma separated list like: "JUSTTIMES,DOUBLES".	Defined Name	Same As ReadEvents Option Code...	"ALL"	GET_ALL	"NEW"	GET_NEW	"SAME"	GET_SAME	"JUSTTIMES"	GET_JUSTTIMES	"DOUBLES"	GET_DOUBLES	"NODATA"	GET_NODATA	"FILTERED"	GET_FILTERED	"ORDERED"	GET_ORDERED
Defined Name	Same As ReadEvents Option Code...																			
"ALL"	GET_ALL																			
"NEW"	GET_NEW																			
"SAME"	GET_SAME																			
"JUSTTIMES"	GET_JUSTTIMES																			
"DOUBLES"	GET_DOUBLES																			
"NODATA"	GET_NODATA																			
"FILTERED"	GET_FILTERED																			
"ORDERED"	GET_ORDERED																			

Returns:

Long see ReadEvents, page 31

Sample Code

Description: This code sample reads up to 1000 'Snip' events for channel 13 in Block-45.

MATLAB `TT.ConnectServer('Local','MyClient')`
 `TT.OpenTank('SomeTank','R')`
 `TT.SelectBlock('Block-45')`
 `NumRecs=TT.ReadEventsV(1000,'Snip',13,0,0,0,0)`
 `TT.CloseTank`
 `TT.ReleaseServer`

Related Calls: [ReadEventsSimple](#), [ReadWavesOnTimeRangeV](#)

ReadWavesOnTimeRangeV

Description: This call returns events that occur in valid time ranges. (See GetValidTimeRangesV, page 46, and example below). Note that waves are built from the events. The events are returned in a variant with each column representing a valid time range. Note that only one channel can be processed at a time, because the events for a single channel are returned in a 2-D matrix.

Prototype: `Function ReadWavesOnTimeRangeV(EventName As String, Channel As Long) As Variant`

Arguments:

String *EventName* four letter event name

Long *Channel* channel number

Also uses the following Global parameters: T1, T2, FillItem, FillValue, WaveSF, SortCode, WavesMemLimit

If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.

Returns:

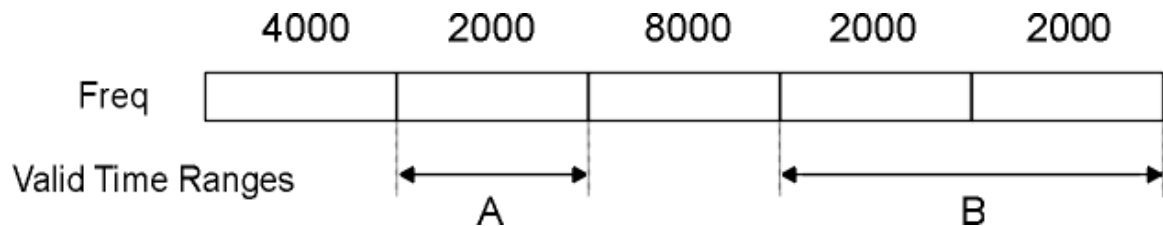
Variant Variant with each column representing the events in a single valid interval of time. Will return only one channel at a time. If one of the valid time ranges is longer than the others, it will fill zeros at the end of the others. In other words, the number of rows of the returned variant will be determined by the length of the longest valid time interval.

Sample Code

Description: The **Sample Code** will return a matrix with two columns, one for valid time range A and the second for B. The first column will be filled with zeroes at the end, so that it is the same length as the second column, which contains a longer valid time range. To get back all the epoch periods as separate columns, use GetEpocsExV.

MATLAB

```
filt = TT.SetFilterWithDescEx('Freq=2000')
waves = TT.ReadWavesOnTimeRangeV('Snip', 1)
```



Related Calls: [GetEpocsExV](#), [GetValidTimeRangesV](#)

ReadWavesOnTimeRangeB

Description: ReadWavesOnTimeRangeB is identical to ReadWavesOnTimeRangeV but allows input of the BSTR String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype:

```
Function ReadWavesOnTimeRangeB(EventName As String,
Channel As Long) As Variant
```

Arguments:

BSTR String *EventName* four letter event name

Long *Channel* channel number

Also uses the following Global parameters: T1, T2, FillItem, FillValue, WaveSF, SortCode, WavesMemLimit

If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.

Returns:

Variant Variant with each column representing the events in a single valid interval of time. Will return only one channel at a time. If one of the valid time ranges is longer than the others, it will fill zeros at the end of the others. In other words,

the number of rows of the returned variant will be determined by the length of the longest valid time interval.

Related Calls: [ReadWavesOnTimeRangeV](#)

ReadWavesV

Description: This call returns a waveform(s) in a variant. ReadWavesV constructs a continuous waveform from a series of events. The waveform is constructed along a timeline with zeroes filling in the samples where no event was acquired. For example, if the event specified contains neural spikes, it will not just concatenate the spikes, but will put the spikes in the correct points of time along the timeline with zeroes filling in between the spikes.

Note that the timeline is not in terms of time, but in samples. The timeline is constructed using the sampling frequency of the event itself, unless otherwise specified by setting the global parameter WaveSF or WaveSFEvent. In case of a scalar event, a sampling frequency of 100Hz is used unless otherwise specified. This ability to set a sampling frequency allows two different event waveforms to be plotted along the same timeline.

Although this call has only one argument, it is affected by a number of global parameters such as T1 (start time), T2 (stop time), Channel, SortCode, and Options (all events, filtered, or new).

If a channel number is specified, then it returns a single column array. If channel=0 (all channels), it returns a variant with each column representing a single channel.

Note that ReadWavesV will not work with a filter that contains 'chan' or 'sort'. For example: it works when the filter = 'freq=1000 and time<20'.

However, it does not work when filter = 'freq=1000 and time<20 and chan<1'. Note that the channel and sort code can be specified using SetGlobalV.

This call returns data in a form that is consistent with ReadEventsV. Waveforms are saved in the same format with events and can be parsed with ParseEvV.

Prototype: `Function ReadWavesV(EventName As String) As Variant`

Arguments:

String *EventName* four letter event name

Also uses the following Global parameters: Channel, SortCode, T1, T2, Options, FillItem, FillValue, WaveSFEvent, WaveSF, WavesMemLimit

If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.

Returns:

Variant Variant with waveforms. If a channel number is specified, then it returns a single column array. If channel=0 (all channels), it returns a variant with each column representing a single channel.

Sample Code

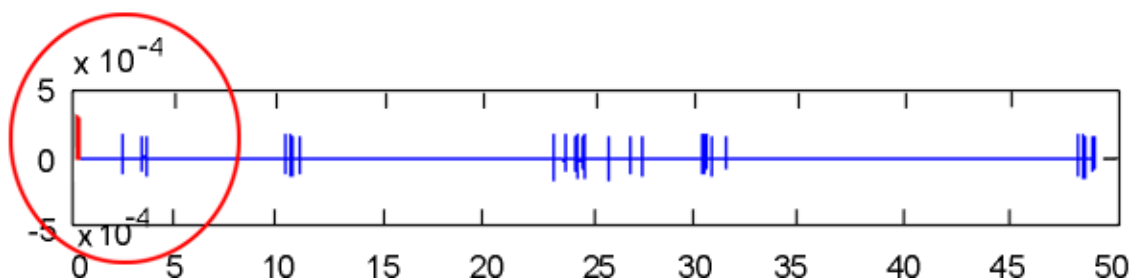
Description: This code sample sets a filter and plots the events that pass the filter.

```
MATLAB      TT.SetGlobalV('Channel', 1);
             TT.SetFilterWithDescEx('Freq=2000');
             wave = TT.ReadWavesV('Snip');
```

```

freq = TT.ReadWavesV('Freq');
plot(wave);
hold on;
plot(freq);

```



Notice that Snip and Freq are not on the same timeline, resulting in a clustering of Freq epochs at the start of the timeline.

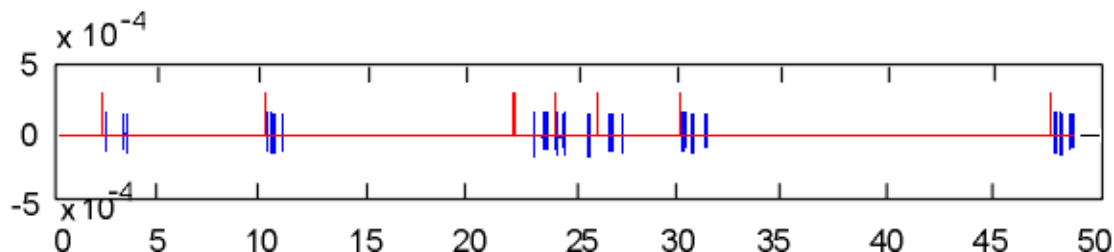
MATLAB

```

TT.SetGlobals('WaveSFEEvent = Snip; Channel = 1');
TT.SetFilterWithDescEx('Freq=2000');
wave = TT.ReadWavesV('Snip');
freq = TT.ReadWavesV('Freq');
plot(wave);
hold on;
plot(freq);

```

This code sample uses SetGlobals to modify the WaveSFEEvent and Channel global parameters, ensuring that the events can be plotted on the same timeline.



Related Calls: [ReadWavesOnTimeRangeV](#), [ReadEventsSimple](#)

ParseEv

Description: Use the ParseEv call to retrieve some or all information for event records cached in local memory by a call to ReadEvents or ReadEventsV. The RecIndex parameter is used to specify the record to access and is zero based. The function will return zero when the RecIndex is specified beyond the end of the returned list.

Prototype: `Function ParseEv(RecIndex As Long, TimeStamp As Double, Channel As Long, SortCode As Long, Npts As Long, pData As Single) As Long`

Arguments:

Long	<i>RecIndex</i>	index of record to retrieve, starting with 0 and indexing up
Double*	<i>FAR* TimeStamp</i>	pointer to a double variable to store time stamp, 0 to avoid having time stamp returned
Long*	<i>FAR* Channel</i>	pointer to a long variable to store channel number, 0 to avoid having channel returned
Long*	<i>FAR* SortCode</i>	pointer to a long variable to store sort code value, 0 to avoid having sort code returned If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.
Long*	<i>FAR* Npts</i>	pointer to a long variable to store the npts in any attached waveform data, 0 to avoid having this data returned
Float*	<i>FAR* pData</i>	pointer to a memory buffer to store the raw waveform

Returns:

Long fails (0), succeeds (1)

Related Calls: [ReadEvents](#) , [ReadEventsV](#)

ParseEvInfoV

Description: ParseEvInfoV and ParseEvV combine to offer a variant compatible version of ParseEv. Review the ParseEv call for more information. This call is used to retrieve information, read using ReadEvents or ReadEventsV, from TTankX's local cache. Any or all event information (but not waveform data) can be retrieved with this call. Using the RecIndex and nRecs parameters you can retrieve information about a number of records with just one call.

Prototype: `Function ParseEvInfoV(RecIndex As Long, nRecs As Long, nItem As Long) as Variant`

Arguments:

Long	<i>RecIndex</i>	starting index of record(s) for which information is to be retrieved (0 based)
Long	<i>nRecs</i>	number of records for which information is to be retrieved The number specified is the number of rows returned. Use 0 to have data returned in a single dimensional array or as a scalar. If RecIndex + nRecs exceeds the end of the cached records, the extra rows will be returned with zeros.
Long	<i>nItem</i>	item code for the information item to be returned Use 0 to have all items returned as columns in the order shown below or select one of the following:

Item Code	Returns	Item Code	Returns
1	size of waveform data in bytes	8	data format code
2	event type	9	waveform sample rate in

			Hz (requires attached waveform data)
3	event code	10	not used (returns 0)
4	channel number	11	X dimension filter ID
5	sorting number	12	Y dimension filter ID
6	time stamp	13	Z dimension filter ID
7	scalar value (valid when no waveform data is attached)	14	fill Item

Returns:

Variant *variant form of data* The variant form of the data is a matrix with the columns being the data item or items and rows being the indexed records. The exact format of the returned data is dependent on which arguments are passed as 0s.

The possible return scenarios are:

- { nRecs > 0 and nItem > 0 } returns a row matrix containing the requested value for nRecs records
- { nRecs > 0 and nItem = 0 } returns a row/column matrix with nRecs rows by 10 columns containing all the information values
- { nRecs = 0 and nItem > 0 } returns a single scalar with the specified value for the specified record index
- { nRecs = 0 and nItem = 0 } returns a row matrix containing the 10 data items for the specified record index

Sample Code

Description: Creates an index of Block-45 from MyTank and reads up to 1000 of the Snip events from time 0 to 47. The ParseEvInfoV returns the time stamp of each event.

MATLAB `TT.ConnectServer('Local','Me')`
`TT.OpenTank('MyTank','R')`
`TT.SelectBlock('Block-45')`
`Nrecs=TT.ReadEventsV(1000,'Snip',0,0,0,47,'ALL')`
`TimeStamps=TT.ParseEvInfoV(0,Nrecs,6)`
`% to get the channel number for record index 17 call`
`chan = TT.ParseEvInfoV(17, 0, 4)`
`% to get all the information items for index 17 call`
`info = TT.ParseEvInfoV(17, 0, 0)`
`% to get all the info for all the records call`
`allinfo = TT.ParseEvInfoV(0, Nrecs, 0)`

Related Calls: [ParseEv](#), [ParseEvV](#), [ReadWavesV](#), [ReadEventsSimple](#)

ParseEvV

Description: ParseEvInfoV and ParseEvV combine to offer a variant compatible version of ParseEv. Review the ParseEv call for more information. This call is used to retrieve waveform data, read using ReadEvents or ReadEventsV, from TTankX's local cache. Using the RecIndex and nRecs parameters you can retrieve waveform data for a number of records with just one call.

Prototype: `Function ParseEvV(RecIndex As Long, nRecs As Long) As Variant`

Arguments:

Long	<i>RecIndex</i>	see ParseEvInfoV
Long	<i>nRecs</i>	number of records for which waveform data is to be retrieved
		Pass 0 or 1 to get a single row of data for a single record.

Returns:

Variant	<i>variant form of data</i>	The variant form of the data is a matrix with the columns being the waveform data and the rows being the indexed records. If nRecs = 0 the waveform data is returned in a row array.
---------	-----------------------------	--

Sample Code

Description: Creates an index of Block-45 from MyTank and reads up to 1000 of the Snip events from time 0 to 47. ParseEvV retrieves the data for the first 10 records.

MATLAB

```

TT.ConnectSever('Local','Me')
TT.OpenTank('MyTank','R')
TT.SelectBlock('Block-45')
Nrecs = TT.ReadEventsV(1000,'Snip',0,0,0,47,'ALL')
WaveData = TT.ParseEvV(0,10)

```

Related Calls: [ParseEv](#), [ParseEvInfoV](#), [ReadWavesV](#), [ReadEventsSimple](#)

SetUseSortName

Description: SetUseSortName sets the sort file used for OpenDeveloper calls that retrieve events like ReadEventsV. The sort code file will be set if the event name matches and the desired channel has a sort named sortID. If this function is not used, the event name does not match, or of the sort name sortID is not present, this function has no effect and the original sort file from the online tank sort is used.

Prototype: `Function SetUseSortName(sortID As String)As Long`

Arguments:

Double	<i>sortID</i>	sort ID given in OpenSorter (the original online tank sort is always named TankSort)
--------	---------------	--

Returns:

Long fails (0), succeeds (1)

Sample Code

Description: The Matlab code below will work to read Snip events of channel = 1 and sort code = 1 from the sort set saved as "Sort1".

MATLAB

```
SetSort1 = TT.SetUseSortName('Sort1')
Filter = TT.SetFilterWithDescEx('sort=1')
AllSort1 =
TT.ReadEventsV(1000,'Snip',1,0,0.0,0.0,'FILTERED')
```

The following example assumes a tank with online sort code named TankSort and sorts generated in OpenSorter named Sort1, Sort2 and Sort3.

For a hypothetical data set, the TankSort sort file has 100 Snip events for each sort code; 1, 2 and 3. Since a different sorting criterion was used for Sort1, this sort file has 50 Snip events for sort code 1 and 2 and 200 events for sort code 3.

If ReadEventsV (Snip, sort code 1) is called before applying SetUseSortName, 100 events will be returned. If SetUseSortName (Sort1) is called before calling ReadEventsV (Snip, sort code 1), the sort code file for Sort1 will be applied and 50 events will be returned. If using a SortCode generated by OpenSorter, see Known Anomalies, page 113.

Epochs and Filtering

TTank X

About Epochs and Filtering

TTankServer uses a process called filtering to perform record querying. With typical database engines SQL or a similar language is used to query records from a larger record set. TTankServer employs a parametric filtering methodology that offers a powerful querying capability that is very fast.

Once an Epoch Index has been created (using CreateEpochIndexing) filtering calls can be made to limit the records (events) returned by TTankServer (ReadEventsV). By default no filters are applied, meaning all valid event records are returned when ReadEvents is called.

Epoch events are defined using an epoch event code or a four-letter epoch event name. Several of the filter methods utilize the event code while others use the epoch event name. The methods GetEventCodes, StringToEvCode, and CodeToString are used to convert or retrieve an event code for use with the TTankX control filters.

TTankX Control Filter Methods

[SetFilter](#)

[SetEpochTimeFilter](#)

[SetEpochTimeFilterV](#)

[SetFilterArray](#)

[SetFilterWithDesc](#)

[SetFilterWithDescEx](#)

Filtering Examples

[Using Epochs to Filter Data](#)

[Reconstructing Waveforms from Events](#)

[Using Filter Arrays](#)

CreateEpocIndexing

Description: A memory based epoch index must be created for the selected block before a client applications can take advantage of high speed data indexing and filtering capabilities. After selecting a block for access, using SelectBlock, call CreateEpocIndexing to instruct TTankServer to build the epoch index. This call must be made each time a new block is selected.

A tilde (~) in front of the block name of the SelectBlock method will automatically generate an epoch index for that block.

Prototype: `Function CreateEpocIndexing() As Long`

Returns:

Long fails (0), succeeds (1)

Sample Code

Description: This code sample indexes all the epoch events for the selected block.

MATLAB `TT.ConnectServer('Local','Me')`
 `TT.OpenTank('MyTank','R')`
 `TT.SelectBlock('Block-45')`
 `TT.CreateEpocIndexing`
 `% Tilde Example`
 `TT.ConnectServer('Local','Me')`
 `TT.OpenTank('MyTank','R')`
 `TT.SelectBlock('~Block-45')`

Related Calls: [ClearEpocIndexing](#), [SelectBlock](#)

ClearEpocIndexing

Description: ClearEpocIndexing clears the epoch indexing information from memory. This function is not often called by the user, it is called automatically when a new block is selected or the tank is closed.

Prototype: `Function ClearEpocIndexing()`

Related Calls: [CreateEpocIndexing](#)

GetEpocCode

Description: GetEpocCode can be used to build a list of epoch code strings currently in the memory index. The memory index must be built using CreateEpocIndexing. You must first call using an index of 0, to get the first epoch code, then increase the index until null is returned.

Prototype: `Function GetEpocCode(Index AS Long) As String`

Arguments:

Long *Index* index number of the epoch code

Returns:

String epoch code string, such as SwpN or FREQ

Related Calls: [CreateEpocIndexing](#)

GetEpocsV

Description: GetEpocsV returns epoch event information from a time region of the block. This variant form includes a list of four doubles that represent the values of the epoch, the start time, the stop time, and the filter status. The size of the variant in bytes is 32 times the number of epochs returned. MaxEpocs determines the maximum number of epochs to be returned. If the number of epochs is greater than the MaxEpocs only the maximum number will be returned.

Prototype: `Function GetEpocsV(TankCode As String, T1 As Double, T2 As Double, MaxEpocs As Long) As Variant`

Arguments:

String	<i>TankCode</i>	event name for the epoch
Double	<i>T1</i>	specifies the starting time in the block Only epochs with start times equal to or greater than T1 will be returned. Use 0 to get all epochs from the start of the block.
Double	<i>T2</i>	specifies the maximum time to return Only epochs with a start time less than T2 will be returned. Use 0 to get all epochs in a block.
Long	<i>MaxEpocs</i>	maximum number of epochs to return Also uses the following Global parameters: RepectOffsetEpoc

Returns:

VARIANT	<i>row of doubles</i>	row of values with information about each epoch returned The values are ordered like: [Epoch Value][Start Time][Stop Time][Filter Selection] ... next epoch For ONset strobe epocs the start times are returned. For OFFset strobe epocs the stop times are returned. For buddy epocs, the start times and stop times are returned.
---------	-----------------------	---

Sample Code

Description: Creates an index of Block-45 from MyTank and gets up to the first 1000 frequency Epochs.

```
MATLAB      TT.ConnectServer('Local','Me')
             TT.OpenTank('MyTank','R')
             TT.SelectBlock('Block-45')
             TT.CreateEpocIndexing
             MyEpocs = TT.GetEpocsV('Freq',0,0,1000)
```

Related Calls: [GetEpocsExV](#), [GetValidTimeRangesV](#), [QryEpocAtV](#)

GetEpocsExV

Description: This call will return the valid epoch events that pass through any preceding filters. It will return a part of, or the entire epoch duration, depending on the mode argument.

Prototype: `Function GetEpocsExV(EpocName As String, Mode As Long) As Variant`

Arguments:

String	<i>EpocName</i>	four letter epoch name
Long	<i>Mode</i>	specifies which part of the epoch to return:
	0	all epochs whose onsets occur within the filter epoch
	1	all epochs, occurring at least in part within the filter epochs
	2	only the parts of the epochs that occur within the filter epochs

Also uses the following Global parameters: MaxReturn, RespectOffsetEpoc, T1, T2

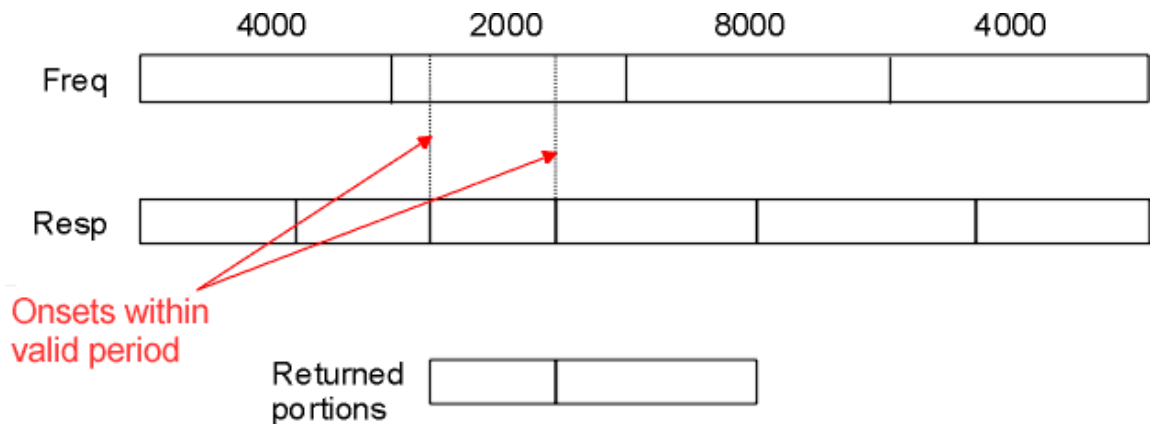
Returns:

Variant
Variant with each column representing a valid epoch. The first row of each column denotes the value of the epoch. The second and third rows contain the start and stop time of each valid epoch duration. Note that these start and stop times may vary based on the mode specified in the arguments.

Sample Code

Description: This code sample sets a filter of Freq=2000 and returns only those epochs whose onsets occur within the valid epoch.

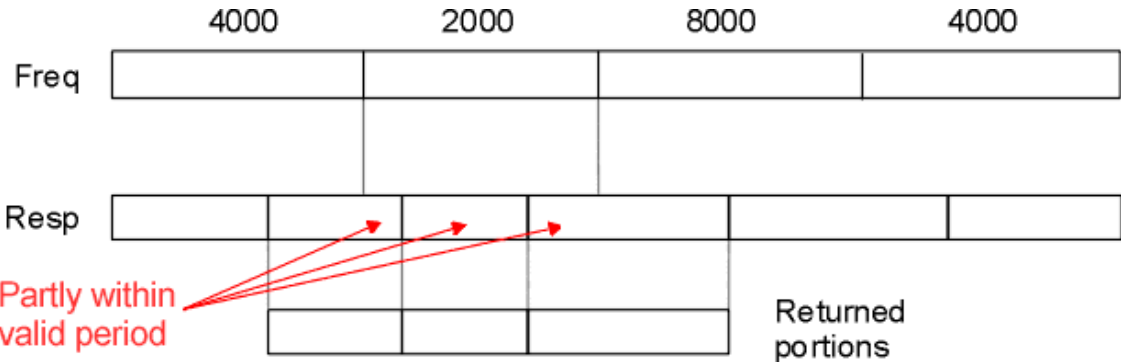
MATLAB `filt = TT.SetFilterWithDescEx('Freq=2000');
response = TT.GetEpocsExV('Resp', 0);`



Again, a filter of Freq=200 is set. However, this time the mode is set to return only those parts of the epochs, which occur within the valid epochs.

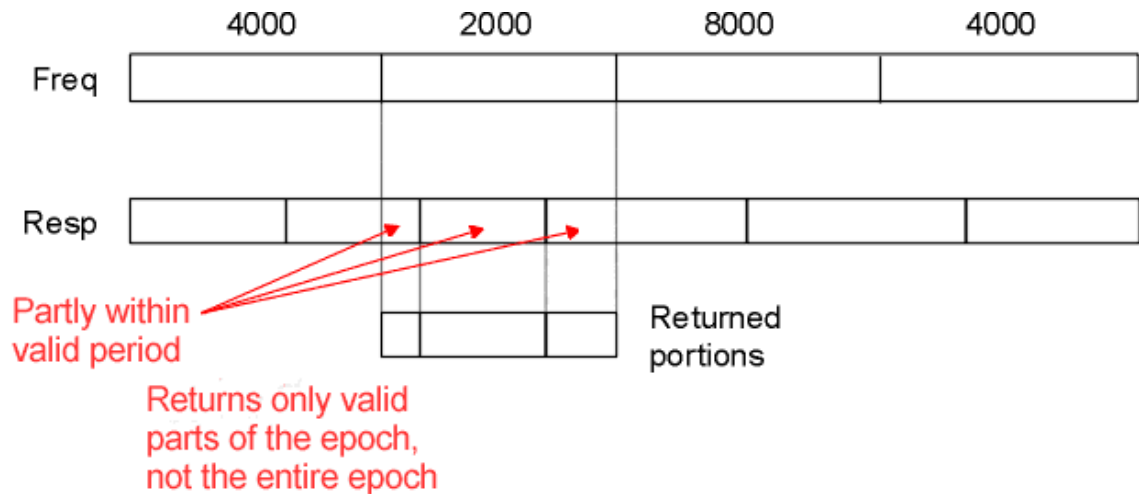
MATLAB `filt = TT.SetFilterWithDescEx('Freq=2000');`

```
response = TT.GetEpocsExV('Resp',1);
```



In this case, only those epochs, which occur at least in part within the valid epochs are returned.

```
MATLAB    filt = TT.SetFilterWithDescEx('Freq=2000');
           response = TT.GetEpocsExV('Resp',2);
```



Related Calls: [CreateEpocIndexing](#), [GetValidTimeRangesV](#)

GetFilterTolerance

Description: Returns the tolerance of the filter. The tolerance is the margin of error allowed in evaluating the conditions of the filter. For example, if filter tolerance is set to 0.001, and the filter is specified as Freq=2000, any epoch with values between 1998 and 2002 will pass through the filter. The default value is 1e-7.

Prototype: `Function GetFilterTolerance() As Long`

Returns:

Long tolerance of the filter

Will return -1 in the absence of a tolerance. For example, if a tank has not been accessed.

Sample Code

Description: This code sample returns the current filter tolerance.

MATLAB `tolerance = TT.GetFilterTolerance`

Related Calls: [SetFilterTolerance](#)

GetValidTimeRangesV

Description: This call returns the valid time ranges based on the preceding filters. If no filters are specified, then it will return the entire duration of the block as a single valid time range.

Prototype: `Function GetValidTimeRangesV() As Variant`

This method uses the following Global parameters: MaxReturn, RespectOffsetEpoc

Returns:

Variant The time ranges are returned in the form of a variant with each column representing a single valid time range. There are always two rows: the first row containing the start time of the valid time range, and the second row containing its stop time.

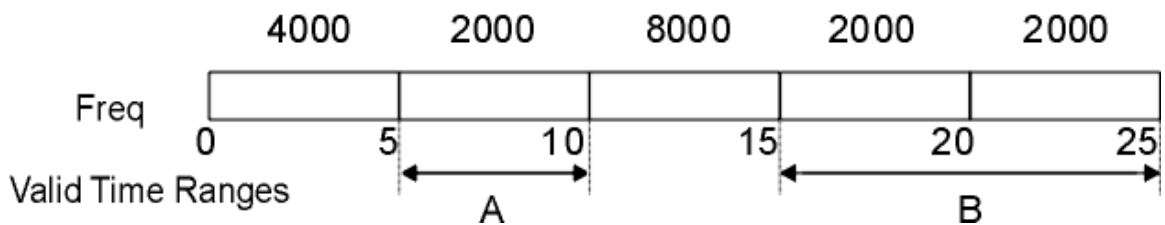
Sample Code

Description: This code sample will return a variant with two columns, each column containing the start and stop times for a valid time range.

MATLAB `filt = TT.SetFilterWithDescEx('Freq=2000')`

`tranges = TT.GetValidTimeRangesV`

`tranges =`
 5 15
 10 25



Related Calls: [GetEpocsExV](#), [ReadWavesOnTimeRangeV](#)

QryEpocAt

Description: QryEpocAt finds the epoch event that is active during the time point specified and returns the specified value associated with that epoch. One of four values is returned: the value of the epoch, the start of the epoch event, the end of the

epoch event, or the filter status. CreateEpocIndexing must be called before QryEpocAt.

Prototype: `Function QryEpocAt(TankCode As Long, rTime As Double, ReqItem As Long, RetVal As Double) As Long`

Arguments:

Long	<i>TankCode</i>	epoch event, a four byte number
Double	<i>rTime</i>	requested time, time at which active epoch is to be found
Long	<i>ReqItem</i>	requested item type: 0 epoch value 1 start time of epoch 2 stop time of epoch 3 filter status (0 or 1)
Long	<i>FAR* RetVal</i>	pointer to the variable that stores the returned value from the index

Returns:

Long fails (0), succeeds (1)

Related Calls: [CreateEpocIndexing](#)

QryEpocAtV

Description: QryEpocAtV works the same as QryEpocAt except that it allows TankCode to be specified with a string and the requested item is returned as a variant.

Prototype: `Function QryEpocAtV(TankCode As String, rTime As Double, ReqItem As Long) As Variant`

Arguments:

String	<i>TankCode</i>	four character epoch name
Double	<i>rTime</i>	requested time, time at which active epoch is to be found
Long	<i>ReqItem</i>	requested item type: 0 epoch value 1 start time of epoch 2 stop time of epoch 3 filter status (0 or 1)

Returns:

Variant *epoch items* returns one of four items: value, start, stop, or filter status of epoch event (see ReqItem in the argument statement above)

Sample Code

Description: Creates an index of Block-45 from MyTank and queries the value of epoch FREQ at time 12.45 seconds.

MATLAB `TT.ConnectServer('Local','Me')`
`TT.OpenTank('MyTank','R')`
`TT.SelectBlock('Block-45')`

```
TT.CreateEpocIndexing
```

```
Epoch = TT.QryEpocAtV('FREQ', 12.45 , 0)
```

Related Calls: [GetEpocsExV](#), [GetValidTimeRangesV](#), [QryEpocAt](#)

ResetFilters

Description: Resets the filters to no filtering. This function must be called whenever a new filtering criterion is to be invoked. For example, if you want all records concurrent with epoch FREQ = 4000 to be returned first and then all event records concurrent with FREQ > 8000 to be returned second, you must call ResetFilters in between the two accesses or all records with either of these criteria true will be returned on the second access.

Refer to the About Epochs and Filtering section, page 41, for more information and filtering examples.

Prototype: `Function ResetFilters()`

Related Calls: [SetFilterWithDesc](#), [SetFilterWithDescEx](#), [SetFilterArray](#), [SetEpocTimeFilterV](#)

SetEpocTimeFilter

Description: SetEpocTimeFilter sets a filter based on the specified offset and duration. The time filter is applied relative to the onset of the specified epoch. The functioning of this call is affected by the RespectOffsetEpoc global parameter. By default, it will respect the duration of buddy epochs. So, if the duration specified is more than the length of the buddy epoch, events that occurred after the offset of the buddy epoch will not pass the filter. To ignore the buddy epoch offset, set RespectOffsetEpoc to 0.

Refer to the About Epochs and Filtering section, page 41, for more information and filtering examples.

Prototype: `Function SetEpocTimeFilter(EpocCode As Long, Offset As Double, Duration As Double) As Long`

Arguments:

Long	<i>EpocCode</i>	event code of an epoch event
Double	<i>Offset</i>	time in seconds from the onset of the epoch to the onset of the filter (can be a negative value)
Double	<i>Duration</i>	duration of the filter in seconds (0 specifies the entire duration of the epoch)

Also uses the following Global parameters: AutoRefEpoch, RespectOffsetEpoc

Returns:

Long fails (0), succeeds (1)

Sample Code

Description: SetEpochTimeFilter and SetEpochTimeFilterV are the same except that one uses the epoch code and one uses the epoch name. See SetEpochTimeFilterV, page 49, for Sample Code.

Related Calls: [SetRefEpoc](#)

SetEpochTimeFilterV

Description: SetEpochTimeFilterV sets a filter based on the specified offset and duration. The time filter is applied relative to the onset of the specified epoch. The functioning of this call is affected by the RespectOffsetEpoc global parameter. By default, it will respect the duration of buddy epochs. So, if the duration specified is more than the length of the buddy epoch, events that occurred after the offset of the buddy epoch will not pass the filter. To ignore the buddy epoch offset, set RespectOffsetEpoc to 0.

Refer to About Epochs and Filtering for more information and filtering examples, page 41.

Prototype: `Function SetEpochTimeFilterV(EpocName As String, Offset As Double, Duration As Double) As Long`

Arguments:

String	<i>EpocName</i>	four letter epoch name
Double	<i>Offset</i>	time in seconds from the onset of the epoch to the onset of the filter (can be a negative value)
Double	<i>Duration</i>	duration of the filter in seconds (0 specifies the entire duration of the epoch)

Also uses the following Global parameters: AutoRefEpoch, RespectOffsetEpoc

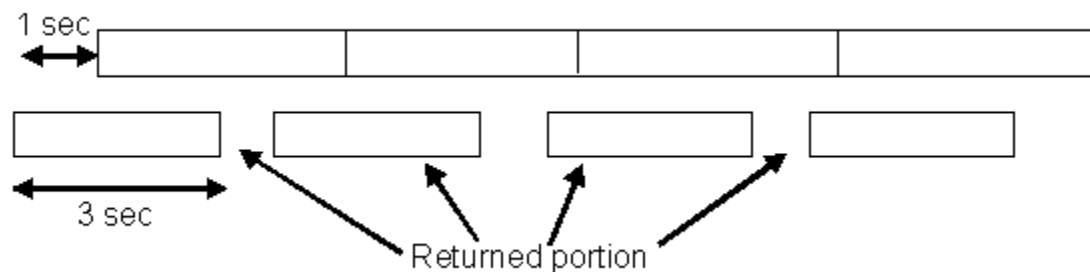
Returns:

Long fails (0), succeeds (1)

Sample Code

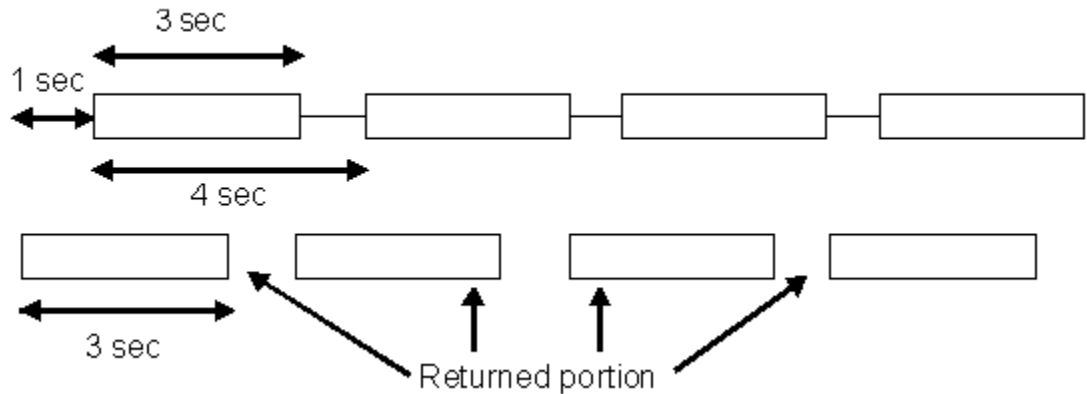
Description: This code sample implements a filter that begins one second before the epoch onset and has a duration of three seconds. Note that in this case, zeroes will fill the one second interval before the start of the data.

MATLAB `Filt = TT.SetepochTimeFilterV('Freq', -1, 3)`



MATLAB `Filt = TT.SetepochTimeFilterV('Freq', -1, 4)`

In this case, because the duration of the buddy epoch is three seconds, the filter passes only three of the four seconds specified. To include the entire four seconds, the `RespectOffsetEpoc` parameter must be set to 0. Also, note that four seconds is the maximum that can be obtained, because that is the duration between the onset of adjacent epochs.



Note: To return the filtered data, a record retrieving call (such as `ReadEventsV` or `ReadWavesV`) must be used with the `Options` global parameter set to `FILTERED`.

Related Calls: [SetRefEpocV](#), [GetEpocsExV](#), [GetValidTimeRangesV](#)

SetEpocTimeFilterB

Description: `SetEpocTimeFilterB` is identical to `SetEpocTimeFilterV` but allows input of the `BSTR` String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype: `Function SetEpocTimeFilterB(EpocName As String, Offset As Double, Duration As Double) As Long`

Arguments:

BSTR String	<i>EpocName</i>	four letter epoch name
Double	<i>Offset</i>	time in seconds from the onset of the epoch to the onset of the filter (can be a negative value)
Double	<i>Duration</i>	duration of the filter in seconds (0 specifies the entire duration of the epoch)

Also uses the following Global parameters: `AutoRefEpoch`, `RespectOffsetEpoc`

Returns:

Long fails (0), succeeds (1)

Related Calls: [SetEpocTimeFilterV](#)

SetFilter

Description: SetFilter is used to filter out events associated with specified epoch blocks. Once a TestCode has been specified, Values V1 and V2 can be used to set boundaries for filtering out epoch events.

SetFilter can be used to filter events associated with specified epoch blocks and can be made multiple times. Multiple calls to SetFilter are cumulative and are logically ORed if applied to the same epoch or logically ANDed if applied across different epochs. To reset all filters make a call to ResetFilters.

Refer to About Epochs and Filtering, page 41, for more information and filtering examples.

Prototype: `Function SetFilter(TankCode As Long, TestCode As Long, V1 As Double, V2 As Double) As Long`

Arguments:

Long	<i>TankCode</i>	epoch event code as 4 bytes long, determined using StringToEvCode or GetEventCodes
Long	<i>TestCode</i>	a single value that sets the criteria value for the filter

letters shown correspond to their ASCII value

associated numbers for use with MATLAB are:

'E'	69	equal to
'N'	78	not equal to
'G'	71	greater than or equal
'L'	76	less than or equal
'A'	65	above, greater than
'B'	66	below, less than
'I'	73	include, between these values
'O'	79	outside of those values

Double	<i>V1</i>	primary value used in an equation
--------	-----------	-----------------------------------

Double	<i>V2</i>	secondary value used an equation
--------	-----------	----------------------------------

The V2 value is used with "I" and "O" to define the range of the filter.

Returns:

Long	number of epoch blocks removed by filtering
------	---

Sample Code

Description: This code sample sets a filter for `FREQ = 2000`.

```
MATLAB
TT.SelectBlock('Block-45')
TT.CreateEpocIndexing
ecode = TT.StringToEvCode('FREQ')
TT.SetFilter(ecode, 69, 2000, 0)
```

Related Calls: [GetEventCode](#), [GetEpocCode](#), [CreateEpocIndexing](#)

SetFilterArray

Description: Assigns a filter along one of three dimensions, and gives it an ID along that dimension. The dimensions are usually assigned the number 0, 1, or 2. They are often also referred to as X, Y, and Z dimensions.

The IDs along these dimensions are called X-ID, Y-ID, and Z-ID respectively. An event that meets the criteria set by one of these filters is assigned the appropriate filter ID. Multiple instances of this call will generate an array of filters, which can then be used to sort and display data.

This method includes an exclusive flag, allowing the user to determine whether events can be assigned IDs for multiple filters. If the exclusive flag is enabled (1), only the lowest ID number will be used for each event.

Refer to About Epochs and Filtering, 41, for more information and filtering examples.

Prototype: `Function SetFilterArray(Dimension As Long, ID As Long, Conditions As String, Exclusive Flag As Boolean) As Long`

Arguments:

Long	<i>Dimension</i>	dimension for which the filter is set; specified as 0, 1, or 2 for X, Y, or Z respectively
Long	<i>ID</i>	filter ID, specified as any number from 1 to 256
String	<i>Conditions</i>	string defining the filter, for example: 'Freq=4000 and Level=2' Note: Boolean operators 'and' and 'or' are allowed within the filter string.
Boolean	<i>Exclusive Flag</i>	flag specifying whether an event which meets the criteria for multiple filters is assigned to more than one ID 1 = exclusive (first ID) 0 = not exclusive (multiple ID's)

Also uses the following Global parameters: RespectOffsetEpoC

Returns:

Long fails (0), succeeds (1)

Sample Code

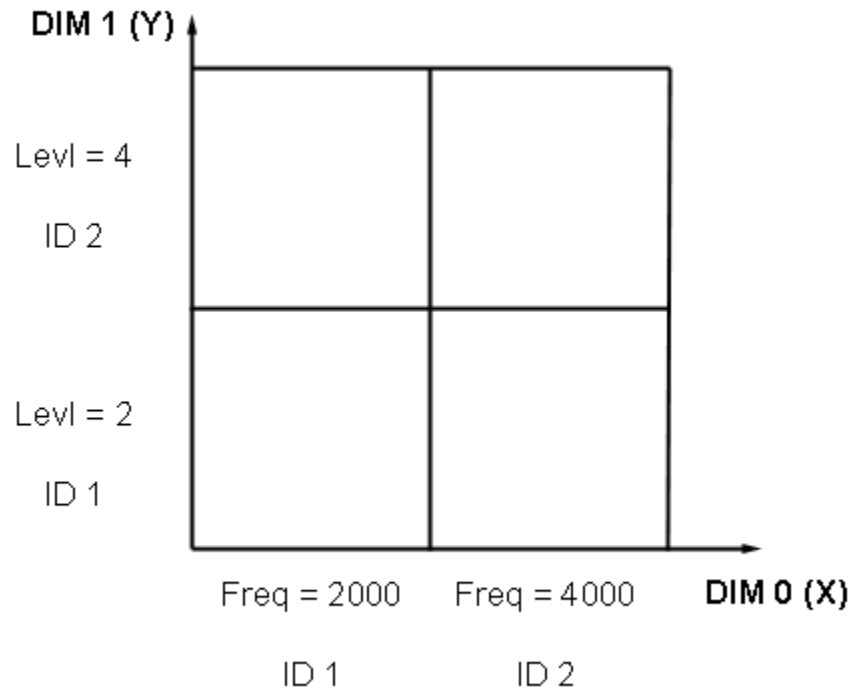
Description: This sample code sets up an array of non-exclusive filters along the X and Y dimensions. Note that X, and Y dimensions are denoted by 0 and 1 respectively. After events have passed through the filter array, their X and Y dimension filter ID properties will be set according to the filter arguments.

MATLAB

```
a = TT.SetFilterArray(0,1,'Freq=2000',0)
b = TT.SetFilterArray(0,2,'Freq=4000',0)
c = TT.SetFilterArray(1,1,'Levl=2',0)
d = TT.SetFilterArray(1,2,'Levl=4',0)
```

When data is read, an event will be returned only if it passes filters specifying at least one of the cells of a grid (as pictured below). In other words, if an event passes the filters Freq=4000 and Levl=2, then it will be assigned the X ID 2, and Y ID 1 and will be returned in the corresponding cell. If an event passes Levl=4,

but no other filter, then it will not be returned. If an event fits into multiple cells of the filter array, then it will be read multiple times, each time with a different set of X and Y IDs. Note that this is not possible when the exclusive flag is set to 1. In that case, the event would be returned only once, with the lowest set of IDs possible.



Related Calls: [SetFilterWithDescEx](#), [SetEpocTimeFilterV](#), [GetValidTimeRangesV](#)

SetFilterWithDesc

Description: SetFilterWithDesc functions the same as SetFilter except that the filter is specified as a string rather than four longs. Refer to SetFilter, page 51, for more information.

A filter description contains three parts, (1) epoch name, (2) operation specification and (3) the value(s). The format looks like: [EpochName] [OperationSpec] [Value(s)].

EpochName -- The epoch name is specified as four chars, such as FREQ, SwpN, or Puff. There are three special keywords that will invoke special non-epoch driven filtering, they are: TIME, CHAN, and SORT. These three keywords allow for full filtering function on the time stamp, channel number, and sort code for event records.

OperationSpec -- Each of the operations enumerated in the SetFilter description has a corresponding text character specification. These characters are '=', '<>', '>=', '<=', '>', and '<'. They meaning of each is based on their use in standard mathematical equations. One exception is that a value range, or the 'include,

between' function, is specified using the '=' character. For example, `FREQ = 1000:8000` is used to specify all FREQs between 1000 and 8000. To specify the 'outside' or not between function use the '<>' characters in the form: `FREQ <> 1000:8000`.

Value(s) -- The values parameter is always a decimal number, such as 12.3 or 768. If two numbers are needed (for specifying a range) use a colon between them, for example 4:44.

Refer to About Epochs and Filtering, page 41, for more information and filtering examples.

Prototype: `Function SetFilterWithDesc (FiltDesc As String) As Long`

Arguments:

String *FilterDesc* see description above

Returns:

Long number of epoch blocks that met the filter condition

Sample Code

Description: This code sample sets filters to select event records concurrent with `|EyeX| < 1.0` and `|EyeY| < 1.0`

MATLAB `TT.SelectBlock('MyBlock-45')`
 `TT.CreateEpocIndexing`
 `TT.SetFilterWithDesc('EyeX = -1.0:1.0')`
 `TT.SetFilterWithDesc('EyeY = -1.0:1.0')`

Related Calls: [SetFilterWithDescEx](#), [SetFilterArray](#), [SetEpocTimeFilterV](#), [SetFilter](#)

SetFilterWithDescEx

Description: Sets multiple, logically related filters in a single string. If multiple calls are made, then the last call will overwrite the previous filters. Different filter characteristics can be logically chained together using ANDs and/or ORs to achieve the desired filter.

Refer to the About Epochs and Filtering page 41, section for more information and filtering examples.

Prototype: `Function SetFilterWithDescEx (Conditions As String) As Long`

Arguments:

String *Conditions* string defining the filter, for example: 'Freq=4000 and Lev1=2'

Note: boolean operators 'and' and 'or' are allowed within the filter string as well as standard filters, such as 'chan' and 'sort' are also allowed.

Also uses the following Global parameters: `RespectOffsetEpoc`

Returns:

Long fails (0), succeeds (1)

Sample Code

Description: This code sample uses a single string to set a filter for stimulus frequency, acquisition channel number, and stimulus level.

MATLAB `Filter = TT.SetFilterWithDescEx('Freq=4000 and chan<5 and Lev1=2')`

Related Calls: [SetFilterArray](#), [SetEpocTimeFilterV](#), [GetValidTimeRangesV](#)

SetFilterTolerance

Description: Sets the tolerance of the filter. The tolerance is the margin of error allowed in evaluating the conditions of the filter. For example, if filter tolerance is set to 0.001, and the filter is specified as Freq=2000, any epoch with values between 1998 and 2002 will pass through the filter. The default value is 1e-7.

Refer to the About Epochs and Filtering page 41, section for more information and filtering examples.

Prototype: `Function SetFilterTolerance(Tolerance As Double) As Long`

Arguments:

Double	<i>Tolerance</i>	tolerance of filters
--------	------------------	----------------------

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

Sample Code

Description: This code sample sets the filter tolerance to 0.00001.

MATLAB `tolerance = TT.SetFilterTolerance(0.00001)`

Related Calls: [GetFilterTolerance](#)

SetRefEpoc

Description: This call will set a reference epoch, such that all events subsequently returned will have time stamps relative to the onset of the specified epoch. This is particularly useful for plotting histograms. This method can be called by the user, but it is also called automatically by the SetEpocTimeFilter method.

When using SetEpocTimeFilter, the AutoRefEpoch global parameter (enabled by default) will cause the epoch by SetRefEpoc to be overwritten by the epoch set by SetEpocTimeFilter. To prevent this, set AutoRefEpoch to 0. Also note that, if a negative offset has been specified in the SetEpocTimeFilter arguments, it is possible to get a negative time stamp value.

These calls are typically used before ReadEvents and do not affect the results of the GetValidTimeRanges or GetEpocsEx calls.

Prototype: `Function SetRefEpoc(EpocCode As Long) As Long`

Arguments:

Long	<i>EpocCode</i>	event code of an epoch event
------	-----------------	------------------------------

Returns:

Long fails (0), succeeds (1)

Sample Code

Description: This code sample sets a reference epoch using the epoch's numeric code.

MATLAB `a = TT.SetRefEpoc(4367)`

Related Calls: [SetEpocTimeFilter](#)

SetRefEpocV

Description: This function will set a reference epoch, such that all events subsequently returned will have time stamps relative to the onset of the specified epoch. This is particularly useful for plotting histograms. This method can be called by the user, but it is also called automatically by the SetEpocTimeFilterV method.

When using SetEpocTimeFilterV, the AutoRefEpoch global parameter (enabled by default) will cause the epoch set by SetRefEpocV to be overwritten by the epoch set by SetEpocTimeFilterV. To prevent this, set AutoRefEpoch to 0. Also note that, if a negative offset has been specified in the SetEpocTimeFilterV arguments, it is possible to get a negative time stamp value.

These calls are typically used before ReadEventsV and do not affect the results of the GetValidTimeRangesV or GetEpocsExV calls.

Prototype: `Function SetRefEpocV(EpocName As String) As Long`

Arguments:

String *EpocName* four letter name of an epoch event

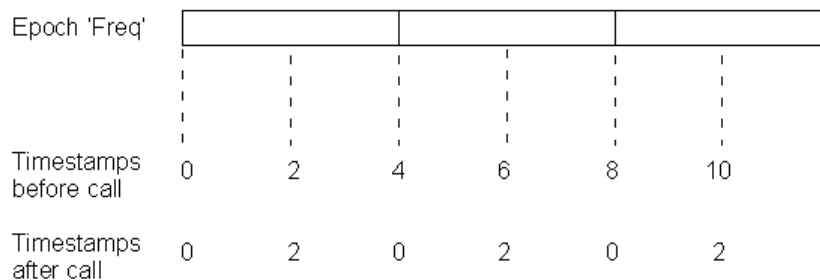
Returns:

Long fails (0), succeeds (1)

Sample Code

Description: This code sample specifies the Freq epoch as the reference epoch. The time stamps of all events returned after this call will be relative to this epoch, as seen in the diagram below.

MATLAB `a = TT.SetRefEpocV('Freq')`



Related Calls: [SetEpocTimeFilterV](#)

SetRefEpocB

Description: SetRefEpocB is identical to SetRefEpocV but allows input of the BSTR String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype: `Function SetRefEpocB(EpocName As String) As Long`

Arguments:

BSTR String	<i>EpocName</i>	four letter name of an epoch event
-------------	-----------------	------------------------------------

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

Related Calls: [SetRefEpocV](#)

Information Access

TTank X

CurBlockMemo

Description: CurBlockMemo returns the memo associated with the currently selected block. If no memo was specified, a null string is returned.

Prototype: `Function CurBlockMemo() As String`

Returns:

String	<i>block memo</i>	returns memo in string or null string if no block is currently selected or if no memo was specified when the block was created
--------	-------------------	--

Sample Code

Description: This code sample returns the memo associated with the currently selected block.

MATLAB `blkmemo = TT.CurBlockMemo`

CurBlockName

Description: CurBlockName returns the name of the currently selected block.

Prototype: `Function CurBlockName() As String`

Returns:

String	<i>block name</i>	returns block name or null string if no block is currently selected
--------	-------------------	---

Sample Code

Description: This code sample returns the name of the currently selected block.

MATLAB `blkmemo = TT.CurBlockName`

CurBlockNotes

Description: CurBlockNotes returns notes associated with the currently selected block. The notes for each store in the block include: the store name, number of points, sample frequency, number of channels and other information.

Prototype: `Function CurBlockNotes() As String`

Returns:

String	<i>StoreName</i>	name of each store
	<i>Enabled</i>	enable status of store
	<i>CircType</i>	circuit type
	<i>NumChan</i>	number of channels
	<i>StrobeMode</i>	onset/offset strobe
	<i>StrobeBuddy</i>	buddy epoch if applicable
	<i>SecTag</i>	secondary tag information if applicable
	<i>NumPoints</i>	number of points
	<i>DataFormat</i>	data format (0: float, 1: 32-bit integer, 2: short, 3: byte)
	<i>SampleFreq</i>	sample frequency

Sample Code

Description: This code sample returns notes associated with the currently selected block.

MATLAB `notes = TT.CurBlockNotes`

CurBlockOwner

Description: CurBlockOwner returns the owner of the currently selected block. If no owner was specified, a null string is returned.

Prototype: `Function CurBlockOwner() As String`

Returns:

String	<i>block owner</i>	returns owner in string or null string if no block is currently selected or if no owner was specified when the block was created
--------	--------------------	--

Sample Code

Description: This code sample returns the owner of the currently selected block.

MATLAB `blkowner = TT.CurBlockOwner`

CurBlockStartTime

Description: CurBlockStartTime returns the start time of the selected block in seconds. The returned value is the elapsed time in seconds from 12:00 AM January 1st, 1970 to the start of the block. Pass the result through FancyTime to convert the result into a date/time string.

Prototype: `Function CurBlockStartTime() As Double`

Returns:

String *start time* returns block start time in seconds

Sample Code

Description: This code sample returns the current block start time and then passes the result through FancyTime to return a more readable value.

MATLAB `start = TT.CurBlockStartTime`
 `formstart = TT.FancyTime(start , 'D/O/Y H:M:S.U')`

Related Calls: [CurBlockStopTime](#), [FancyTime](#)

CurBlockStopTime

Description: CurBlockStopTime returns the stop time of the selected block in seconds. The returned value is the elapsed time in seconds from 12:00 AM January 1st, 1970 to the end of the block. Pass the result through FancyTime to convert the result into a date/time string.

Prototype: `Function CurBlockStopTime() As Double`

Returns:

String *stop time* returns block stop time in seconds

Sample Code

Description: This code sample returns the current block stop time and then passes the result through FancyTime to return a more readable value.

MATLAB `stop = TT.CurBlockStopTime`
 `formstop = TT.FancyTime(stop , 'D/O/Y H:M:S.U')`

Related Calls: [CurBlockStartTime](#), [FancyTime](#)

FancyTime

Description: FancyTime converts a time in double format to string format based on the user's specifications. The input argument Time is assumed to be the total elapsed time from 12:00 AM January 1st, 1970 up to the event of interest.

Prototype: `Function FancyTime(Time as Double, Format as String)`
 `As String`

Arguments:

Double *Time* Tank Time in double Format
String *Format* Format for returned value

Year	Month	Day	Hours	Minutes	Seconds	frac/Sec	D of W
Y	O	D	H	M	S	U	W

Returns:

String *Time* Time in string format with user specified formatting

Sample Code

Description: This code sample returns the time in the format 'Date, Time, Day of Week' e.g. '01/Dec/2005 10:04:23.63 Fri' Using characters such as '/', ':', '.' and ' ' further delineate the string.

MATLAB `start = TT.CurBlockStartTime`
 `formstart = TT.FancyTime(start , 'D/O/Y H:M:S.U W')`

GetCodeSpecs

Description: GetCodeSpecs (get code specifications) queries the block and returns the event record specifications for the event code specified. If successful a number of properties within the TTankX control are assigned values as follow:

EvChannel -- channel for first record found
EvDataSize -- size of record in 32 bit hunks
EvDForm -- waveform data format (see technical specs)
EvSampFreq -- sampling frequency of waveform data
EvType -- record type code

Prototype: `Function GetCodeSpecs(EvCode As Long) As Long`

Arguments:

Long *EvCode* event code in long format

Returns:

Long fails (0), succeeds (1)

Related Calls: [ParseEvInfoV](#)

GetEnumServer

Description: GetEnumServer returns servers that are enumerated (registered) on your computer. 0 is returned when no more servers are found. Use this function to build a list of enumerated servers on your computer. To get the first server (typically 'Local') use an index of 0. Then increase the index until null is returned.

Prototype: `Function GetEnumServer(Index As Long) As String`

Arguments:

Long *Index* index of enumerated server, index start at zero

Returns:

String *server name* name of server indexed

Sample Code

Description: This code sample gets the server name at index zero.

MATLAB `servname = TT.GetEnumServer(0)`

Related Calls: [GetEnumTanks](#), [QueryBlockName](#), [GetHotBlock](#)

GetEnumTank

Description: GetEnumTank is used to build a list of tanks enumerated (registered) on the connected server. To get the first tank use an index of 0. Then call with increasing indexes until null is returned.

Prototype: `Function GetEnumTank(Index As Long) As String`

Arguments:

Long *Index* position in the registry, index starts at zero

Returns:

String *tank name* name of the tank at the index specified, null string indicates the end of enumerated tanks

Sample Code

Description: This code sample gets the tank at index 0 of the registry.

MATLAB `TT.GetEnumTank(0)`

Related Calls: [GetEnumServer](#), [QueryBlockName](#), [GetHotBlock](#)

GetError

Description: GetError retrieves any pending error string or null if there is no error pending.

Prototype: `Function GetError() As String`

Returns:

String *error message* error message string or null

Sample Code

Description: This code sample checks for pending error.

MATLAB `if TT.OpenTank('MyTank','R')==0
 errmess = TT.GetError
end`

GetEventCodes

Description: Use this call to build a list of valid event codes for the selected block. GetEventCodes returns a list of longs containing event codes for the event type specified.

Prototype: `Function GetEventCodes (EvType As Long) As Variant`

Arguments:

Long	<i>EventType</i>	event type code or 0 for all event types.
------	------------------	---

Returns:

VARIANT	<i>row of longs</i>	lists of all the codes
---------	---------------------	------------------------

Related Calls: [SetFilter](#)

GetGlobalStringV

Description: This call will return the current string value of the specified [global parameter](#). Note that this call supports only string parameters.

Prototype: `Function GetGlobalStringV(GlobalName As String) As String`

Arguments:

String	<i>GlobalName</i>	global parameter name
--------	-------------------	-----------------------

Returns:

String	current value of the specified global parameter
--------	---

Sample Code

Description: This code sample returns the value of the global parameter, Options.

MATLAB `TT.GetGlobalStringV('Options')`

Related Calls: [GetGlobalV](#), [SetGlobalV](#), [SetGlobalStringV](#), [SetGlobals](#), [ResetGlobals](#)

GetGlobalStringB

Description: GetGlobalStringB is identical to GetGlobalStringV but allows input of the BSTR String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype: `Function GetGlobalStringB(GlobalName As String) As String`

Arguments:

BSTR String	<i>GlobalName</i>	global parameter name
-------------	-------------------	-----------------------

Returns:

String	current value of the specified global parameter
--------	---

Related Calls: [SetGlobalStringV](#), [GetGlobalStringV](#)

GetGlobalV

Description: This call will return the current value of the specified global parameter.

Prototype: `Function GetGlobalV (GlobalName As String) As Long`

Arguments:

String	<i>GlobalName</i>	global parameter name
--------	-------------------	-----------------------

Returns:

Long	current value of the specified global parameter
------	---

Sample Code

Description: This code sample returns the current value for the global parameters, Channel and T2.

MATLAB

```
TT.GetGlobalV('Channel')
TT.SetGlobalV('T2')
```

Related Calls: [SetGlobalV](#), [SetGlobalStringV](#), [SetGlobals](#), [ResetGlobals](#)

GetGlobalB

Description: GetGlobalB is identical to GetGlobalV but allows input of the BSTR String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype: `Function GetGlobalB (GlobalName As String) As Long`

Arguments:

BSTR String	<i>GlobalName</i>	global parameter name
-------------	-------------------	-----------------------

Returns:

Long	current value of the specified global parameter
------	---

Related Calls: [SetGlobalV](#), [SetGlobalStringV](#), [SetGlobals](#), [ResetGlobals](#)

GetHotBlock

Description: Returns the block that is being recorded into the opened tank. If no block is open for recording a null string is returned.

Prototype: `Function GetHotBlock() As String`

Returns:

String	<i>block name</i>	name of block being recorded into
--------	-------------------	-----------------------------------

Sample Code

Description: This code sample connects to the local server, opens the tank, "MyTank" for reading and returns the block name of any block that is being recorded into MyTank.

MATLAB `TT.ConnectServer('Local','Me')`
 `TT.OpenTank('MyTank','R')`
 `recblock = TT.GetHotBlock`

Related Calls: [GetEnumTanks](#), [GetEnumServer](#), [QueryBlockName](#)

GetStatus

Description: GetStatus can be used to obtain information about an open tank.

Prototype: `Function GetStatus(StatCode As Long) As Long`

Arguments:

Long *StatCode* status code for item to be retrieved

Defined Variable	Function	Integer
STAT_TANKSTATE	tank state (R,W,M,C)	0
STAT_CACHEUSAGE	percentage of cache in use	1
STAT_CACHEDEPTH	amount of memory allocated in cache	2
STAT_CACHECOLLIDE	number of collisions in the cache	3
STAT_ORDERERROR	number of ordering errors	4
STAT_EVRATE	event rate (number of events stored to tank per second)	5
STAT_DATARATE	data rate (number of bytes stored to tank per second)	6

Returns:

Long *return value* requested value or -1 if operation failed

Sample Code

Description: This code sample returns the event rate for the opened tank.

MATLAB `TT.SelectBlock('Block-1')`
 `evrate = TT.GetStatus(5)`

GetTankItem

Description: GetTankItem returns the path or the tank version for the tank provided in TankName. This function is only valid with enumerated or registered tanks.

Prototype: `Function GetTankItem(TankName as String, ItemCode as String) As String`

Arguments:

String	<i>TankName</i>	Name of the enumerated tank
String	<i>ItemCode</i>	PT returns path to tank VERSION returns the version of the tank or a null string if the tank does not exist

Returns:

String	<i>Path</i> (<i>ItemCode</i> = 'PT')	Path to the given tank
	<i>Version</i> (<i>ItemCode</i> = 'VERSION')	'20' – new format tank '10' – legacy tank null string – tank does not exist

Sample Code

Description: This code sample returns the path to the given tank.

MATLAB `path = TT.GetTankItem('DemoTank2','PT')`

QueryBlockName

Description: QueryBlockName returns the block name for a given block index. This function can be used to build a list of blocks within a tank. The first call must be made with BlockNumber of 0, then the index can be increased until null is returned.

Prototype: `Function QueryBlockName(BlockNumber As Long) As String`

Arguments:

String	<i>BlockNumber</i>	block number (zero based)
--------	--------------------	---------------------------

Returns:

String	<i>block name</i>	name of block, or null string if there are no more block names to get
--------	-------------------	---

Sample Code

Description: This code sample returns the name of block 45 then selects it for access. If there is no block 45 a null string is returned.

MATLAB `blkNme = TT.QueryBlockName(45)
TT.SelectBlock(blkNme)`

Related Calls: [GetEnumTanks](#), [GetEnumServer](#), [GetHotBlock](#)

Misc Utilities

TTank X

CodeToString

Description: CodeToString converts a long event code to a four character string. Use this call and StringToEvCode to convert back and forth from a long number representation of event codes to the four character string representation.

Prototype: `Function CodeToString(EvCode As Long) As String`

Arguments:

Long	<i>EvCode</i>	event code in long format
------	---------------	---------------------------

Returns:

String	<i>string event code</i>	returns a 4 character string for the event code
--------	--------------------------	---

Related Calls: [StringToEvCode](#)

DFromToString

Description: DFromToString converts a data format code to a descriptive string.

Prototype: `Function DFromToString(DFormCode As Long) As String`

Arguments:

Long	<i>DFormCode</i>
------	------------------

<i>Defined Data Format</i>				<i>Format</i>	<i>Value</i>	<i>Returns</i>
DFORM_FLOAT	float	0				"Float"
DFORM_LONG	long	1				"Long"
DFORM_SHORT	short	2				"Short"
DFORM_BYTE	byte	3				"Byte"
DFORM_DOUBLE	double	4				"Double"

Returns:

String	<i>description</i>	returns a string description for the data format (see above)
--------	--------------------	--

EvTypeToString

Description: EvTypeToString returns a string description for event type codes. An event type is the type of event, such as snippet, strobe, or streamed data.

Prototype: `Function EvTypeToString(evTypeCode As Long) As String`

Arguments:

Long	<i>evTypeCode</i>				
		Defined Event Types	Event Type	Hex Value	Returns
		EVTTYPE_UNKNOWN	unknown	0x00000000	"Unknown"
		EVTTYPE_STRON	Strobe ON	0x00000101	"Strobe+"
		EVTTYPE_STROFF	Strobe OFF	0x00000102	"Strobe-"
		EVTTYPE_SCALAR	Scalar	0x00000201	"Scalar"
		EVTTYPE_STREAM	Stream	0x00008101	"Stream"
		EVTTYPE_SNIP	Snip	0x00008201	"Snip"
		EVTTYPE_MARK	Marker	0x00008801	"Mark"
		EVTTYPE_HASDATA	has associated waveform data	0x00008000	"HasData"
Returns:					
String	<i>description</i>				returns a string description for the data format (see above)

GetNPer

Description: GetNPer (get number per long) calculates the size of a data format relative to a long in integer format. For example, a byte would be 4 since 4 bytes equals a long.

Prototype: `Function GetNPer(DForm As Long) As Long`

Arguments:

Long	DForm	Defined Data Format	Format	Value	Returns
		DFORM_FLOAT	float	0	1
		DFORM_LONG	long	1	1
		DFORM_SHORT	short	2	2
		DFORM_BYTE	byte	3	4

Returns:

Long	<i>number per long</i>	returns the number of data elements per long
------	------------------------	--

ResetGlobals

Description: This call will reset all the global parameters to their default values. See list of default values of global parameters.

Prototype: `Function ResetGlobals () As Void`

Returns:

Void Void []

Sample Code

Description: This code sample resets all global parameters to their default settings.

MATLAB `TT.ResetGlobals`

Related Calls: [SetGlobals](#), [SetGlobalStringV](#) , [GetGlobalV](#), [SetGlobalV](#), [GetGlobalStringV](#), [SetGlobals](#)

SetGlobalV

Description: This call will set a global parameter to the value specified. See list of global parameters.

Prototype: `Function SetGlobalV(GlobalName As String, GlobalValue As Long) As Long`

Arguments:

String	<i>GlobalName</i>	global parameter name
Long	<i>GlobalValue</i>	desired value of the global parameter

Returns:

Long fails (0), succeeds (1)
 + *Tip* ... If 0 is returned, the name of the global parameter might have been entered incorrectly.

Sample Code

Description: This code sample sets new values for the global parameters, Channel and T2, then returns the number of events

MATLAB `TT.SetGlobalV('Channel', 1)`
 `TT.SetGlobalV('T2', 10)`
 `events = TT.ReadEventsSimple('Snip')`

Related Calls: [GetGlobalV](#), [SetGlobalStringV](#), [SetGlobals](#), [ResetGlobals](#)

SetGlobalB

Description: SetGlobalB is identical to SetGlobalV but allows input of the BSTR String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype: `Function SetGlobalB(GlobalName As String, GlobalValue As Long) As Long`

Arguments:

BSTR String	<i>GlobalName</i>	global parameter name
Long	<i>GlobalValue</i>	desired value of the global parameter

Returns:

Long fails (0), succeeds (1)
 + *Tip* ... If 0 is returned, the name of the global parameter might have been entered incorrectly.

Related Calls: [SetGlobalV](#)

SetGlobals

Description: This call allows the user to set multiple global parameters of different types in a single call. Each global parameter specified is separated by a semicolon and uses an equal sign to assign its desired value.

Prototype: `Function SetGlobals(Settings As String) As Long`

Arguments:

String *Settings* multiple parameter settings specified as a string

Returns:

Long fails (0), succeeds (1)

Sample Code

Description: This code sample sets several global parameters with a single call.

MATLAB `TT.SetGlobals('Options=FILTERED; Channel=1; T1=1; T2=10')`

Related Calls: [ResetGlobals](#), [SetGlobalStringV](#), [GetGlobalV](#), [SetGlobalV](#), [GetGlobalStringV](#), [ResetGlobals](#)

SetGlobalsB

Description: SetGlobalsB is identical to SetGlobals but allows input of the BSTR String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype: `Function SetGlobalsB(Settings As String) As Long`

Arguments:

BSTR String *Settings* multiple parameter settings specified as a string

Returns:

Long fails (0), succeeds (1)

Related Calls: [SetGlobals](#)

SetGlobalStringV

Description: This call will set the string value of the specified string global parameter. Note that this call supports only string parameters.

Prototype: `Function SetGlobalStringV(GlobalName As String, GlobalValue As String) As Long`

Arguments:

String	<i>GlobalName</i>	global parameter name
String	<i>GlobalValue</i>	desired value of the global parameter

Returns:

Long fails (0), succeeds (1)

If 0 is returned, the name of the global parameter might have been entered incorrectly.

Sample Code

Description: This code sample sets the global parameter, Options, to FILTERED.

MATLAB `TT.SetGlobalStringV('Options', 'FILTERED')`

Related Calls: [GetGlobalV](#), [SetGlobalV](#), [GetGlobalStringV](#), [SetGlobals](#), [ResetGlobals](#)

SetGlobalStringB

Description: SetGlobalStringB is identical to SetGlobalStringV but allows input of the BSTR String type for increased compatibility with languages that have tighter data type restrictions such as Python.

Prototype: `Function SetGlobalStringB(GlobalName As String, GlobalValue As String) As Long`

Arguments:

BSTR String	<i>GlobalName</i>	global parameter name
BSTR String	<i>GlobalValue</i>	desired value of the global parameter

Returns:

Long fails (0), succeeds (1)

If 0 is returned, the name of the global parameter might have been entered incorrectly.

Related Calls: [SetGlobalStringV](#)

StringToEvCode

Description: StringToEvCode converts a four character string to its corresponding event code in long format. This call is used to obtain epoch codes required for SetFilter. This call can be used with CodeToString to go back and forth between formats.

Prototype: `Function StringToEvCode(EvCode As String) As Long`

Arguments:

String	<i>EvCode</i>	event code in four character string format
Returns:		
Long	<i>event code</i>	returns the event code in long value format
Related Calls: CodeToString		

TTankInterfaces

About the TTankInterfaces

TTankInterfaces includes four graphical user interfaces (GUIs) for displaying, modifying, and accessing data tanks through TDT's server applications.

ServerSelect

This interface allows users to modify and access server names.

TankSelect

This interface allows users to modify and access tank names.

BlockSelect

This interface allows users to access and modify blocks within an active tank.

EventSelect

This interface allows users to access event properties.

The operations of these components are linked through their event handlers. Developers can coordinate events fired by individual components to develop interactive applications similar to TDT's OpenScope, a client application in the OpenEx software suite. An example program developed in Visual Basic is included to provide an illustration of how these components are used together.

Properties, Methods, and Events

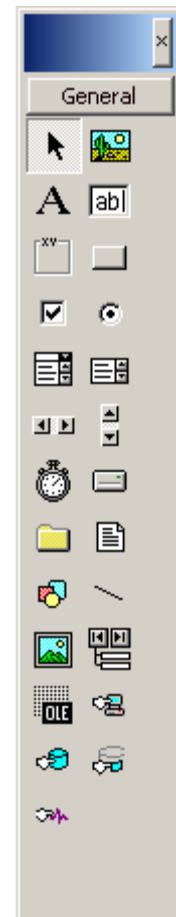
The GUI structure of the TTankInterfaces uses three programming components: properties, methods, and events. Properties describe the way the interface looks and behaves. Methods are programming calls similar to those used with other TDT ActiveX controls. Events are user-triggered behaviors. When an event is triggered a sub-program that executes commands associated with that event is called.

To use the TTankInterfaces in Visual Basic:

1. Add the control by right-clicking the **Control Toolbar** and selecting **Components** from the drop-down box.
2. Next, choose **TTankInterfaces** from the list and click **OK**.

After adding TTankInterfaces to the toolbar you will see icons for ServerSelect, TankSelect, BlockSelect and EventSelect as shown to the right.

3. Double-click the appropriate icon to add the control to the form.



The figure below shows an example program developed in Visual Basic that uses the TTankInterfaces.

ServerSelect

Form1

SERVER:

Local

TANK: @Local...

Name	Path
DEMOTANK	C:\TDT\OpenEx\Tanks
test	C:\ttankinterface\

BLOCK: @Local.DEMOTANK...

Name	Date	Start	Duration	Stop
Block-1	Jul 05, 20...	12:17:25	00:00:11	12:17:3
Block-2	Jul 05, 20...	12:17:38	00:00:00	12:17:3
Block-3	Jul 05, 20...	12:17:42	00:00:01	12:17:4
Block-4	Jul 05, 20...	12:58:05	00:00:15	12:58:2
Block-5	Jul 05, 20...	15:15:43	00:00:08	15:15:5

EVENT: @Local.DEMOTANK.Block-1...

EvID	EvType	DForm	nSize	Fs	First
CSPK	Snip	Float	31	24414.1	00:00:

TankSelect

BlockSelect

EventSelect

ServerSelect

TTankInterfaces

About ServerSelect

The ServerSelect interface can be used to access record information about the TTank servers. A list of the servers is stored in the Windows registry. When ServerSelect is used a list of the servers enumerated in the registry is displayed. A right-click in the ServerSelect interface brings up a menu that allows users to add, select, refresh, or remove a server. Arguments from ServerSelect can then be passed to TankSelect so that TankSelect is updated with the tanks associated with that server.

ActiveServer, Properties

TTankInterfaces.ServerSelect

Description: The ActiveServer property returns the name of the server that is currently selected. This server is called the active server and is denoted in the list of servers with a green arrow. The ActiveServer property can be written to or read.

Prototype: `ServerSelect.ActiveServer [=String]`

Variable:

String name of the active server

Sample Code

Description: This code sample returns/sets the active server for the control.

Visual Basic `Server=ServerSelect.ActiveServer`

AllowEdit, Properties

TTankInterfaces.ServerSelect

Description: The AllowEdit property enables the right-click menu which allows users to modify or add and remove servers from the Windows registry. In general TDT recommends that users add and remove servers from OpenWorkbench, or one of TDT's other OpenEx applications, and use the TTank interfaces to access existing servers and tanks.

Prototype: `ServerSelect.AllowEdit [=Long]`

Sample Code

Description: This code sample sets the AllowEdit property to 0.

Visual Basic `ServerSelect.AllowEdit=0`

AddServer, Methods

TTankInterfaces.ServerSelect

Description: The AddServer method places a key in the Windows registry allowing access to a tank server. Once a server has been added, either locally or over a network, the

tanks on that server are accessible. TDT recommends that users use OpenWorkbench or TankMon to add new servers. Most calls should be made through the default server.

Prototype: `Function AddServer(ServerName As String, IPAddress As String, UserName As String, Domain As String, Password As String) As long`

Arguments:

String	<i>ServerName</i>	name of the local server
String	<i>IPAddress</i>	IP address of the local server
String	<i>UserName</i>	name of the user, must be a user on the server
String	<i>Domain</i>	domain name, if no domain leave blank
String	<i>Password</i>	user's password on the server

Returns:

Long fails(0), succeeds (1)

DeleteServer, Methods

TTankInterfaces.ServerSelect

Description: This method deletes a tank server from the Windows registry. Deleting a server also removes all the information about the tanks on that server. TDT recommends that users use OpenWorkbench or TankMon to delete servers. Most calls should be made through the default server.

Prototype: `Function DeleteServer(ServerName As String) As Long`

Arguments:

String	<i>ServerName</i>	name of the local server
--------	-------------------	--------------------------

Returns:

Long fails (0), succeeds (1)

Refresh, Methods

TTankInterfaces.ServerSelect

Description: This method refreshes the controls and re-enumerates the servers in the Windows registry. Refresh should be called after AddServer or DeleteServer has been called and after the event ServerChanged has occurred.

Prototype: `Function Refresh()`

ServerChanged, Events

TTankInterfaces.ServerSelect

Description: A ServerChanged event occurs when an inactive server is selected. For example, when a user clicks on an inactive server the server becomes active and a ServerChanged event has occurred.

Prototype: `Private Function ServerSelect.ServerChanged(NewServer As String)`

Variables:

String	<i>NewServer</i>	name of the server, use either "Local" or "" to set the default server
--------	------------------	--

Sample Code

Description: When the server is changed, that is, selected in the Server Select window; the Windows registry is refreshed and the active server is updated.

Visual Basic

```
Private Sub ServerSelect_ServerChanged(NewServer As String)
    'update the TankSelect component
    TankSelect.UseServer = NewServer
End Sub
```

TankSelect

TTankInterfaces

About TankSelect

TankSelect is used to display the tanks located on a selected server. Only tanks that have been enumerated, that is, added to the Windows registry on that server, are accessible. TankSelect provides the same functionality, for accessing and modifying tanks, that is available in OpenEx applications. TankSelect can take information passed from ServerSelect and use it to find enumerated tanks on the server.

ActiveTank, Properties

TTankInterfaces.TankSelect

Description: The ActiveTank property returns the name of the active tank. The active tank is the name of the chosen tank on the connected server. Users can either set the property or read the property.

Prototype: `TankSelect.ActiveTank [=String]`

Variable:

String	name of the active tank
--------	-------------------------

Sample Code

Description: This code sample returns the active tank.

Visual Basic

```
NewTank=TankSelect.ActiveTank
```

AllowPopUp, Properties

TTankInterfaces.TankSelect

Description: The AllowPopUp property enables a right-click menu which allows users to modify or add and remove tanks from the Windows registry. In general TDT recommends that users add and remove tanks from OpenWorkbench, or one of TDT's other OpenEx applications, then use the TTankInterface to access existing servers, and tanks.

Prototype: `TankSelect.AllowPopUp [=Long]`

Variable:

Long 0 = no pop up modification, 1 = pop up modification

Sample Code

Description: This code sample sets the AllowPopUp property to 0.

Visual Basic `TankSelect.AllowPopUp=0`

ShowServer, Properties

TTankInterfaces.TankSelect

Description: The tank's server name shows up in the top label when this property is true.

Prototype: `TankSelect.ShowServer [=Long]`

Variable:

Long 0 = server not shown, 1 = server displayed

Sample Code

Description: This code sample sets the ShowServer property to 0.

Visual Basic `TankSelect.ShowServer=0`

SingleClickSelect, Properties

TTankInterfaces.TankSelect

Description: If this property is true, single-clicking a tank in the TankSelect control will select the tank. Otherwise, only double-clicking a tank name will select a tank and make it active.

Prototype: `TankSelect.SingleClickSelect [=Long]`

Variable:

Long 0 = double-click required to select tank, 1 = single-click to select tank

Sample Code

Description: This code sample sets the SingleClickSelect property to 0.
Visual Basic `TankSelect.SingleClickSelect=0`

HideDetails, Properties

TTankInterfaces.TankSelect

Description: If this property is true, the tank details will be hidden. Otherwise, the full path name of the tank will be displayed.

Prototype: `TankSelect.HideDetails [=Long]`

Variable:

Long 0 = path details displayed, 1 = path details hidden

Sample Code

Description: This code sample sets the HideDetails property to 0.

Visual Basic `TankSelect.HideDetails=0`

UseServer, Properties

TTankInterfaces.TankSelect

Description: The UseServer property sets that server as the active server and refreshes the ActiveX control to show the enumerated tanks. If a null string is passed, the server is set to the default server, that is "Local".

Prototype: `TankSelect.UseServer [=String]`

Variable:

String server name

Sample Code

Description: This code sample changes the Active Server to MyServer and refreshes the TankSelect control.

Visual Basic `TankSelect.UseServer="MyServer"`

Refresh, Methods

TTankInterfaces.TankSelect

Description: This method refreshes the controls and re-enumerates the tanks in the Windows registry. Refresh should be called after an event. In some cases, such as UseServer, the tank control is automatically refreshed.

Prototype: `Function Refresh()`

TankChanged, Events

TTankInterfaces.TankSelect

Description: A TankChanged event occurs when an inactive tank is selected and active. For example, when a user clicks an inactive tank the tank is selected and becomes active and a TankChanged event has occurred.

Prototype: `Private Function TankSelect.TankChanged(ActTank As String, ActServer As String)`

Variables:

String	<i>ActTank</i>	name of the tank to be activated
String	<i>ActServer</i>	name of the active server

Sample Code

Description: When the Tank is changed, that is, selected in the Tank Select window; the registry is refreshed and the ActiveTank name is updated.

Visual Basic

```
Private Sub TankSelect_TankChanged(ActTank As String,
ActServer As String)
BlockSelect.UseServer = ActServer
BlockSelect.UseTank = ActTank
BlockSelect.Refresh
```

BlockSelect

TTankInterfaces

About BlockSelect

BlockSelect gives users direct control over information about a block. Each block contains information about the duration of the acquisition block as well as its start and end time.

ActiveBlock, Properties

TTankInterfaces.BlockSelect

Description: The ActiveBlock property returns the name of the active block. The active block is the name of the selected block in the active tank. Users can either set the property or read the property.

Prototype: `BlockSelect.ActiveBlock[=String]`

Variable:

String	name of the active block
--------	--------------------------

Sample Code

Description: This code sample returns the active block.

Visual Basic

```
Newblock=BlockSelect.ActiveBlock
```

AllowPopUp, Properties

TTankInterfaces.BlockSelect

Description: The AllowPopUp property enables a right-click menu to allow users to retrieve information about a block, refresh or erase a block.. In general TDT recommends that users add and remove servers and tanks from OpenWorkbench, or TDT's other OpenEx applications, and then use the TTankInterface to access existing servers, tanks, and blocks.s.

Prototype: `BlockSelect.AllowPopUp [=Long]`

Variable:

Long 0 = no pop up modification, 1 = pop up modification

Sample Code

Description: This code sample sets the AllowPopUp property to 0.

Visual Basic `BlockSelect.AllowPopUp=0`

HideDetails, Properties

TTankInterfaces.BlockSelect

Description: When this property is true, block details will be hidden and only block names will be displayed; when it is false, detailed information (date, start time, stop time, duration, owner, and memo) about each block will be available to users depending on the control's other appearance settings.

Prototype: `BlockSelect.HideDetails [=Long]`

Variable:

Long 0 = block settings displayed, 1 = block settings hidden

Sample Code

Description: This code sample sets the HideDetails property to 0.

Visual Basic `BlockSelect.HideDetails=0`

ShowDate, Properties

TTankInterfaces.BlockSelect

Description: Dates will be created in the following format "month day year" and displayed when ShowDate is true. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `BlockSelect.ShowDate [=Long]`

Variable:

Long 0 = hide date, 1 = show date

Sample Code

Description: This code sample displays the date in the BlockSelect window.

Visual Basic `BlockSelect.ShowDate=1`
`BlockSelect.HideDetails=0`

ShowDuration, Properties

TTankInterfaces.BlockSelect

Description: The duration of the block in Hours:Minutes:Second (H:M:S) is displayed for each block when ShowDuration is true. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `BlockSelect.ShowDuration [=Long]`

Variable:

Long 0 = hide duration, 1 = show duration

Sample Code

Description: This code sample does not show the duration of the block.

Visual Basic `BlockSelect.ShowDuration=0`
`BlockSelect.HideDetails=0`

ShowMemo, Properties

TTankInterfaces.BlockSelect

Description: The Memos (user defined memos on the OpenWorkbench Block Info property sheet) for each block are displayed if ShowMemo is true. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `BlockSelect.ShowMemo [=Long]`

Variable:

Long 0 = hide memo, 1 = show memo

Sample Code

Description: The code sample displays the memos recorded on the OpenWorkbench Block Info property sheet.

Visual Basic `BlockSelect.ShowMemo=1`
`BlockSelect.HideDetails=0`

ShowOwner, Properties

TTankInterfaces.BlockSelect

Description: The Owner of the block (defined by the user on the OpenWorkbench Block Info property sheet) for each block is displayed if ShowOwner is true. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `BlockSelect.ShowOwner [=Long]`

Variable:

Long 0 = hide owner, 1 = show owner

Sample Code

Description: The code sample displays the owner of the block.

Visual Basic `BlockSelect.ShowOwner=1`
 `BlockSelect.HideDetails=0`

ShowStart, Properties

TTankInterfaces.BlockSelect

Description: The start time of the block in Hours:Minutes:Second (H:M:S) is displayed for each block when ShowStart is true. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `BlockSelect.ShowStart [=Long]`

Variable:

Long 0 = hide start time, 1 = show start time

Sample Code

Description: The code sample displays the start time of the block.

Visual Basic `BlockSelect.ShowStart=1`
 `BlockSelect.HideDetails=0`

ShowStop, Properties

TTankInterfaces.BlockSelect

Description: The stop time of the block in Hours:Minutes:Second (H:M:S) is displayed for each block when ShowStop is true. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `BlockSelect.ShowStop [=Long]`

Variable:

Long 0 = hide stop time, 1 = show stop time

Sample Code

Description: The code sample displays the stop time of the block.
Visual Basic `BlockSelect.ShowStop=1`
`BlockSelect.HideDetails=0`

ShowTank, Properties

TTankInterfaces.BlockSelect

Description: This property displays the active tank name in the top label of the control when ShowTank is true and HideDetails is set to false.

Prototype: `BlockSelect.ShowTank [=Long]`

Variable:

Long 0 = hide tank name, 1 = show tank name

Sample Code

Description: The code sample displays the name of the active tank.

Visual Basic `BlockSelect.HideDetails=0`
`BlockSelect.ShowTank=1`

SingleClickSelect, Properties

TTankInterfaces.BlockSelect

Description: Single-clicking the control will select a block if this property is true, otherwise only double-clicking can select a tank.

Prototype: `BlockSelect.SingleClickSelect [=Long]`

Variable:

Long 0 = Double-click required to select block, 1 = single-click to select block

Sample Code

Description: The code sample sets the SingleClickSelect property to 0.

Visual Basic `BlockSelect.SingleClickSelect=0`

UseServer, Properties

TTankInterfaces.BlockSelect

Description: The UseServer property sets that server as the active server and refreshes the ActiveX controls to show the enumerated tanks and their blocks. If a null string is passed, the server is set to the default server, that is "Local".

Prototype: `BlockSelect.UseServer [=String]`

Variable:

String server name

Sample Code

Description: The code sample changes the active server to the local server.

Visual Basic `BlockSelect.UseServer="Local"`

UseTank, Properties

TTankInterfaces.BlockSelect

Description: The UseTank property sets the tank as the active tank and refreshes the ActiveX control to show the blocks.

Prototype: `BlockSelect.UseTank[=String]`

Variable:

String server name

Sample Code

Description: The code sample changes the active tank to MyTank.

Visual Basic `BlockSelect.UseTank="MyTank"`

Refresh, Methods

TTankInterfaces.BlockSelect

Description: This property refreshes the controls and finds the blocks. Refresh should be called after an event. In some cases, such as UseServer, the block control is automatically refreshed.

Prototype: `Function Refresh()`

BlockChanged, Events

TTankInterfaces.BlockSelect

Description: A BlockChanged event occurs when an inactive tank is selected and activated. For example, a user clicks on an inactive block.

Prototype: `Private Function BlockSelect.BlockChanged(ActBlock As String, ActTank As String, ActServer As String)`

Variables:

String *ActBlock* name of the block that was activated

String *ActTank* name of the active tank

String *ActServer* name of the active server

Sample Code

Description: When the Block is changed, that is, selected in the Block Select window; that block is selected, the registry is refreshed, and the active tank name is updated.

Visual Basic

```
Private Sub BlockSelect_BlockChanged(ActBlock As
String, ActTank As String, ActServer As String)
'update the EventSelect component
EventSelect.UseServer = ActServer
EventSelect.UseTank = ActTank
EventSelect.UseBlock = ActBlock
EventSelect.Refresh
End Sub
```

EventSelect

TTankInterfaces

About EventSelect

The EventSelect component has been designed to show all the event information in a selected block of the active tank (UseTank) on the active server (UseServer). Its interface contains several properties, methods, and events that affect the component's overall appearance.

ActiveEvent, Properties

TTankInterfaces.EventSelect

Description: The ActiveEvent property returns the name of the active event. The active event is the name of the selected event in the active block. Users can either set or read the property.

Prototype: `EventSelect.ActiveTank [=String]`

Variable:

String name of the active event

Sample Code

Description: The Sample Code returns the active event.

Visual Basic `NewEvent=EventSelect.ActiveEvent`

AllowActive, Properties

TTankInterfaces.EventSelect

Description: The AllowActive property allows users to select an event by single or double-clicking that event. If this is false users can not select an event from the EventSelect interface.

Prototype: `EventSelect.AllowEvent [=Long]`

Variable:

Long 0 = cannot selective event, 1 = events selectable from interface

Sample Code

Description: The Sample Code allows events to be selected from the interface.

Visual Basic `EventSelect.AllowActive=1`

HideDetails, Properties

TTankInterfaces.EventSelect

Description: When this property is true, only event names will be displayed; when it is false, detailed information (such as ID, Type, Data Format, Size, Frequency, and First Time) about each event will be available to the user depending on the control's other appearance settings.

Prototype: `EventSelect.HideDetails [=Long]`

Variable:

Long 0 = event settings hidden, 1 = event settings displayed

Sample Code

Description: The code sample sets the HideDetails property to 0.

Visual Basic `EventSelect.HideDetails=0`

ShowBlock, Properties

TTankInterfaces.EventSelect

Description: The ShowBlock property displays the active block name in the top label of the control when ShowBlock is true and HideDetails is set to false.

Prototype: `EventSelect.ShowBlock [=Long]`

Variable:

Long 0 = hide block name, 1 = show block name

Sample Code

Description: The code sample displays the name of the active block.

Visual Basic `EventSelect.ShowBlock=1`
`EventSelect.HideDetails=0`

ShowDataFormat, Properties

TTankInterfaces.EventSelect

Description: The ShowDataFormat property displays the data format (such as integer, float, or long) when ShowDataFormat and HideDetails are set true.

Prototype: `EventSelect.ShowDataFormat [=Long]`

Variable:

Long 0 = don't show data format, 1 = show data format

Sample Code

Description: The code sample displays the data format of the event.

Visual Basic `EventSelect.HideDetails=1`
`EventSelect.ShowDataFormat=1`

ShowFirstTime, Properties

TTankInterfaces.EventSelect

Description: This property displays the first time the event occurred in the block.

Prototype: `EventSelect.ShowFirstTime [=Long]`

Variable:

Long 0 = don't show first time, 1 = show first time of event

Sample Code

Description: The code sample displays the first time the event occurs in the block.

Visual Basic `EventSelect.HideDetails=1`
`EventSelect.ShowFirstTime=1`

ShowSampleFreq, Properties

TTankInterfaces.EventSelect

Description: This property displays the sample frequency of the event. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `EventSelect.ShowSampleFreq [=Long]`

Variable:

Long 0 = does not show sample frequency, 1 = shows sample frequency

Sample Code

Description: The code sample displays the sample frequency of the event.

Visual Basic `EventSelect.ShowSampleFreq=1`
`EventSelect.HideDetails=0`

ShowServer, Properties

TTankInterfaces.EventSelect

Description: The tank's server name is displayed in the top label of the control when this property is true

Prototype: `EventSelect.ShowServer [=Long]`

Variable:

Long 0 = server not shown, 1 = server displayed

Sample Code

Description: The code sample sets the ShowServer property to 0.

Visual Basic `EventSelect.ShowServer=0`
`EventSelect.HideDetails=0`

ShowSize, Properties

TTankInterfaces.EventSelect

Description: The ShowSize property displays the size of the event in number of samples. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `EventSelect.ShowSize [=Long]`

Variable:

Long 0 = don't show size, 1 = show size

Sample Code

Description: The Sample Code displays the size of the event in number of samples.

Visual Basic `EventSelect.ShowSize=1`
`EventSelect.HideDetails=0`

ShowTank, Properties

TTankInterfaces.EventSelect

Description: This property displays the active tank name in the top label of the control when ShowTank is true and HideDetails is set to false.

Prototype: `EventSelect.ShowTank [=Long]`

Variable:

Long 0 = hide tank name, 1 = show tank name

Sample Code

Description: The code sample displays the name of the active tank.

Visual Basic `EventSelect.HideDetails=0`
`EventSelect.ShowTank=1`

ShowType, Properties

TTankInterfaces.EventSelect

Description: The ShowType property displays the store type (such as Snippet or Stream) when ShowType is true. For the information to display properly, this property should be set while HideDetails is true.

Prototype: `EventSelect.ShowType [=Long]`

Variable:

Long 0 = don't show type, 1= show store type

Sample Code

Description: The code sample displays the store type of the event.

Visual Basic `EventSelect.ShowType=1`
 `EventSelect.HideDetails=0`

SingleClickSelect, Properties

TTankInterfaces.EventSelect

Description: Single-clicking the control will select an event if this property is true, otherwise only double-clicking can select an event.

Prototype: `EventSelect.SingleClickSelect [=Long]`

Variable:

Long 0 = double-click required to select events, 1 = single-click to select events

Sample Code

Description: The Sample Code sets the SingleClickSelect property to 0.

Visual Basic `EventSelect.SingleClickSelect=0`

UseBlock, Properties

TTankInterfaces.EventSelect

Description: The UseBlock property sets the block as the active block and refreshes the ActiveX control to show the events.

Prototype: `EventSelect.UseBlock [=String]`

Variable:

String block name

Sample Code

Description: The code sample changes the active block to MyBlock.

Visual Basic `EventSelect.UseBlock="MyBlock"`

UseServer, Properties**TTankInterfaces.EventSelect**

Description: The UseServer property sets the server as the active server. The default server selection should be "Local".

Prototype: `EventSelect.UseServer [=String]`

Variable:

String server name

Sample Code

Description: The code sample changes the active server to local.

Visual Basic `EventSelect.UseServer="Local "`

UseTank, Properties**TTankInterfaces.EventSelect**

Description: The UseTank property sets that tank as the active tank and refreshes the ActiveX control to show the blocks.

Prototype: `BlockSelect.UseTank [=String]`

Variable:

String server name

Sample Code

Description: Changes the active tank to MyTank.

Visual Basic `EventSelect.UseTank="MyTank"`

Refresh, Methods**TTankInterfaces.EventSelect**

Description: This method refreshes the controls and lists the events. Refresh should be called after an event is changed. In some cases, such as UseEvent, the event control is automatically refreshed.

Prototype: `Function Refresh()`

GetEvName, Methods**TTankInterfaces.EventSelect**

Description: The GetEvName method returns the name of the event, given the index, as a string. The index starts with 1.

Prototype: `Function GetEvName(Index as long) As String`

Arguments:

long	<i>Index</i>	event index
------	--------------	-------------

Returns:

String	Event Name
--------	------------

ActEventChange, Events**TTankInterfaces.EventSelect**

Description: The ActEventChange event is fired only when AllowActive is true. If the user single-clicks the control when SingleClickActive is true or the user double-clicks the control, then its argument is the new acting event name.

Prototype: `Private Function EventSelect.ActEventChange(ActEvent as String)`

Variables:

LPCSTR	<i>ActEvent</i>	name of the event that was activated
--------	-----------------	--------------------------------------

Sample Code

Description: In the Sample Code, when the event is changed a message box pops up with the name of the new event.

Visual Basic

```
Private Sub EventSelect_ActEventChange(NewActEvent As String)
    'output the active event name
    MsgBox (NewActEvent)
End Sub
```

TTankInterfaces Example

This example is installed with OpenEx Software Suite and can be found in the following path:

C:\TDT\OpenEx\Examples\TTankX_Example\Matlab\TTankInterfacesExample\

About the Example

This example demonstrates a simple sequence of connecting the different TTankInterfaces ActiveX controls together so that a change in one interface is passed through all the interfaces. For example, if you change the active tank it updates the block selection window. Selection of a particular block lists the events stored in that block.

The example GUI interface was designed using Matlab's GUI editor. This program runs in Matlab 7 or greater. It uses the four TTankInterfaces: ServerSelect, TankSelect, BlockSelect, and EventSelect.

Note: Matlab contains its own naming scheme for COM objects, thus the ServerSelect, TankSelect, BlockSelect, and EventSelect COM objects are named activex1, activex2, activex3, and activex4 respectively.

The program responds to the following four events: the server is changed, the tank is changed, the block is changed, and/or an event is changed. A RunAnalysis button is included to illustrate how to add TTank function calls to the GUI interface. When pressed, the button calls the ReadEventsSimple function which returns the total number of events for all channels of the currently selected Block and EventID.

Several files are provided for TTankInterfacesExample:

Main.m - This file creates the GUI interface. Run this file in Matlab to run the example.

RunAnalysis.m - This file provides the function call for the RunAnalysis button in the GUI interface.

TTankInterfacesExample.fig - This file defines the GUI interface and contains the COM objects.

Note: The following files are auto generated from the TTankInterfacesDemo.fig file

TTankInterfacesExample.m - Contains the event listener functions for actions that occur in the GUI interface.

TTankInterfacesExample_activex1 - Describes the TTankInterfaces.ServerSelect COM object.

TTankInterfacesExample_activex2 - Describes the TTankInterfaces.TankSelect COM object.

TTankInterfacesExample_activex3 - Describes the TTankInterfaces.BlockSelect COM object.

TTankInterfacesExample_activex4 - Describes the TTankInterfaces.EventSelect COM object.

ServerChanged

When the ServerChanged event occurs the function activex1_ServerChanged is called. This function then calls functions associated with TankSelect, the next interface in the group. TankSelect calls the UseServer function which returns all the tanks on that server and refreshes the TankSelect screen. The code is shown below.

```
function activex1_ServerChanged(hObject, eventdata, handles)

% Process Server selection info for TankSelect
handles.activex2.UseServer = eventdata.NewServer;
handles.activex2.Refresh;

% Update global variable CurrentServer
global CurrentServer;
CurrentServer = eventdata.NewServer;
```

TankChanged

When the TankChanged event occurs, the function activex2_TankChanged is run. This function then calls functions associated with BlockSelect, the next interface in the group. BlockSelect calls the following functions: UseServer, UseTank, and Refresh. These functions return all the blocks on that tank and refresh the BlockSelect screen. In addition, if the current tank is changed, the currently selected block and event are deselected. The code is shown below.

```

function activex2_TankChanged(hObject, eventdata, handles)

% Process Server and Tank selection info for BlockSelect
handles.activex3.UseServer = eventdata.ActServer;

% Deselects the previously selected Block if the current Tank is
% changed
handles.activex3.ActiveBlock = '';
handles.activex3.Refresh;

% Deselects the previously selected Event and clears the event
% list if the current Tank is changed
handles.activex4.UseBlock = '';
handles.activex4.ActiveEvent = '';
handles.activex4.Refresh;

% Update global variable CurrentTank
global CurrentTank;
CurrentTank = eventdata.ActTank;

```

BlockChanged

When the BlockChanged event occurs, the function `activex3_BlockChanged` is run. This function then calls functions associated with EventSelect, the next interface in the group. EventSelect calls the following functions: UseServer, UseTank, UseBlock, and Refresh. These functions return all the events in that block and refresh the EventSelect interface. In addition, if the current block is changed, the currently selected event is deselected. The code is shown below.

```

function activex3_BlockChanged(hObject, eventdata, handles)

% Process Server, Tank, and Block selection info for EventSelect
handles.activex4.UseServer = eventdata.ActServer;
handles.activex4.UseTank = eventdata.ActTank;
handles.activex4.UseBlock = eventdata.ActBlock;

% Deselects the previously selected Event if the current Block is
% changed
handles.activex4.ActiveEvent = '';
handles.activex4.Refresh;

```

ActEventChanged

When the ActEventChange event occurs the function `activex4_ActEventChanged` is run. This function then stores the selected Event in the global variable `CurrentEvent` before calling the refresh function. Once an event has been selected, the RunAnalysis button can be used to return the total number of events from the currently selected Block. The code is shown below.

```

function activex4_ActEventChanged(hObject, eventdata, handles)

% Process Event Selection and refresh
global CurrentEvent;
CurrentEvent = eventdata.NewActEvent;
handles.activex4.Refresh;

```


RunAnalysis

When the RunAnalysis button is pressed, the program RunAnalysis.m is called. This program contains the function ReadEventsSimple which returns the total number of events for the currently selected Block and Event.

```
% Return number of events for the selected Block/Event  
N = TT.ReadEventsSimple(CurrentEvent)
```


TDevAcc

About TDevAcc

TDevAcc is a series of methods for accessing and controlling hardware through an OpenWorkbench server. TDevAcc can be used to develop client applications similar to TDT's OpenController application.

TDevAcc provides access to System 3 real-time processing devices during an experiment. Client applications developed using TDevAcc can control circuit parameters, retrieve information from device buffers, and read device tags in real-time. Keep in mind that this unprotected access must be used carefully. An entire OpenEx experiment could 'crash' if a flawed attempt to access tags is executed. TDevAcc supports a modified tag access protocol similar to the one used by the RPco.X interface. Developers should be very familiar with the RPco.X interface, and RPvdsEx circuit design and use, before attempting to use TDevAcc.

TDevAcc uses targets to implement real-time control. When OpenWorkbench is running, client applications developed using TDevAcc can call the OpenWorkbench server and access OpenWorkbench targets. OpenWorkbench targets include the device name, or the name assigned to a hardware device within OpenWorkbench, and the parameter tag, or the name of a tag created in the circuit in RPvdsEx. The device name and parameter tag are used together and are separated by a period to create a target, such as Amp1.LPFreq. The target identifies and provides access to a specific parameter tag within a circuit running on a specific real-time processing device.

Before using the TDevAcc methods, users should have a strong understanding of RPvdsEx circuits and OpenEx methodology along with a background in programming with TDT ActiveX controls.

Users should be mindful of using good 'closed loop' access when working with TDevAcc. This means always releasing your servers.

A typical server access session for a client consists of five main steps:

1. Run the OpenWorkbench application.
2. Load an OpenWorkbench configuration file.
3. Call ConnectServer -- Called to connect to the OpenWorkbench server. The connection is terminated with CloseConnection.
4. Perform any number of operations with the OpenWorkbench server.
5. Call CloseConnection -- Called to release the OpenWorkbench server.

A standard MATLAB routine might look like the routine below:

```
DA = actxcontrol('TDevAcc.X')
DA.ConnectServer('Local')
    %Your code
DA.CloseConnection
```

Organization of TDevAcc Methods

TDevAcc methods can be divided into three basic groups:

- **Setup and Control** -- The methods in this group are used to setup access to OpenWorkbench and control OpenWorkbench system modes.
- **Hardware Data Access** -- The methods in this group are used to read or write data to hardware device components.
- **Hardware Information Retrieval** -- The methods in this group are used to access information, such as status or sample frequency, about a device.

Setup and Control

TDevAcc X

ConnectServer

Description: ConnectServer initiates a connection with an OpenWorkbench server. The connection adds a client to the server. A project has to be loaded in order for Connect Server to return 1 and it will fail if OpenProject is not loaded or if the loaded OpenProject does not have a valid Workbench configuration file.

Prototype: `Function ConnectServer(ServerName As String) As Long`

Arguments:

String	<i>ServerName</i>	name of the server, 'Local' is most common
--------	-------------------	--

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

Sample Code

Description: This code sample connects to the Local server.

MATLAB

```
DA = actxcontrol('TDevAcc.X')
DA.ConnectServer('Local')
```

CheckServerConnection

Description: CheckServerConnection verifies if OpenWorkbench is loaded and if the OpenWorkbench server has loaded a circuit file to the OpenWorkbench application. The method will return a 1 if OpenWorkbench is correctly loaded and configured and in Standby, Preview, or Record mode. It returns 0 if OpenWorkbench is not loaded, configured, or if OpenWorkbench is in Idle mode.

Prototype: `Function CheckServerConnection() As Long`

Returns:

Long	Idle (0), Standby (1), Preview (1), or Record (1)
------	---

Sample Code

Description: This code sample checks the connection to the server and returns a message if the client is not connected to the server.

```
MATLAB    DA.ConnectServer('Local')
           if DA.CheckServerConnection==0
               display('Client application not connect to server')
           end
```

GetSysMode

Description: GetSysMode (get system mode) returns the state of OpenWorkbench as a long. This call can be used in conjunction with SetSysMode to control the operational mode of your entire OpenEx system. The various modes of OpenWorkbench, including Idle, Standby, Preview, and Record; are described in the OpenEx Manual.

Prototype: `Function GetSysMode() As Long`

Returns:

long system mode: Idle (0), Standby (1), Preview (2), or Record (3)

Sample Code

Description: This code sample opens a connection to the OpenWorkbench server. If the OpenWorkbench mode is Record (3) the routine is run.

```
MATLAB    if DA.ConnectServer('Local')==1 then
           if DA.GetSysMode==3 then
               %Start Routine
           end
           end
```

SetSysMode

Description: SetSysMode (set system mode) sets the state of OpenWorkbench through the system mode. The possible modes include: Idle, Standby, Preview, and Record.

Prototype: `Function SetSysMode(NewMode As Long) As Long`

Arguments:

Long *NewMode* sets the mode of the system: Idle (0), Standby (1), Preview (2), and Record (3)

Returns:

Long fails(0), succeeds (1)

Sample Code

Description: This code sample opens a connection to the OpenWorkbench server. If the OpenWorkbench mode is not Record (3) mode, SetSysMode places the OpenWorkbench in Record (3) mode.

```
MATLAB    if DA.ConnectServer('Local')==1
           if DA.GetSysMode ~= 3
               DA.SetSysMode(3)
           end
           end
```

SetTankName

Description: SetTankName sets the active tank if OpenWorkbench is loaded and in Idle or Standby mode. If you are setting the value to a registered tank, you need only provide the tank name for the argument TankName. Otherwise, provide the entire path to the tank.

Prototype: `Function SetTankName(TankName as String) As Long`

Returns:

Long fails(0), succeeds (1)

Sample Code

Description: This code sample opens a connection to the OpenWorkbench server and sets the active tank to the registered tank DemoTank2.

MATLAB `DA.ConnectServer('Local')`
 `DA.SetTankName('DemoTank2')`

Note: to use an unregistered tank, use the absolute path to the tank

`DA.SetTankName('C:\TDT\OpenEx\Tanks\DemoTank2')`

GetTankName

Description: GetTankName returns name of the active tank if OpenWorkbench is loaded. If the tank has not yet been specified in OpenWorkbench, GetTankName returns the null string.

Prototype: `Function GetTankName() As String`

Returns:

String name of the active tank

Sample Code

Description: This code sample opens a connection to the OpenWorkbench server and gets the name of the active tank.

MATLAB `DA.ConnectServer('Local')`
 `DA.GetTankName`

CloseConnection

Description: CloseConnection closes the connection to the OpenWorkbench server.

Prototype: `Function CloseConnection()`

Sample Code

Description: This code sample opens a connection to the OpenWorkbench server then closes it after the client application is finished.

MATLAB `DA.ConnectServer('Local')`
`% Your Code`
`DA.CloseConnection`

Hardware Data Access

TDevAcc X

SetTargetVal

Description: SetTargetVal (set target value) sends a value to a target and is used to modify a parameter tag within an RCO circuit. See About TDevAcc, page 97 for more information on targets.

Prototype: `Function SetTargetVal(Target As String, Val As Double) As Long`

Arguments:

String	<i>Target</i>	target name in DevName.TagName format
Double	<i>Val</i>	value to assign to the target

Returns:

Long	fails(0), succeeds (1)
------	------------------------

Sample Code

Description: This code sample sets the value of target Acq1.Thresh to 5.

MATLAB `DA.SetTargetVal('Acq1.Thresh', 5)`

GetTargetVal

Description: GetTargetVal retrieves a target value and is used to read a parameter tag within an RCO circuit. See About TDevAcc, page 97 for more information on targets.

Note: this function will return 0.0 if the target specified is invalid or cannot be read. You may want to validate the target using GetTargetType before accessing it with GetTargetVal.

Prototype: `Function GetTargetVal(Target As String) As Double`

Arguments:

String	<i>Target</i>	target name in DevName.TagName format
--------	---------------	---------------------------------------

Returns:

Double	returns the value read from the target
--------	--

Sample Code

Description: This code sample returns the cycle usage of device Acq1.

MATLAB `DA.GetTargetVal('Acq1.zCycUse')`

WriteTarget

Description: WriteTarget is used to send data to a memory buffer located on a processor device.

WriteTarget functions similarly to WriteTargetVEX but is designed for Legacy users. New users should refer to the WriteTargetVEX function.

Note: The floating point data array pData must be cast as a single for use with Matlab.

Prototype: `Function WriteTarget(Target As String, nOS As Long, nWords As Long, pData As Single) As Long`

Arguments:

String	<i>Target</i>	name of parameter tag.
Long	<i>nOS</i>	offset within buffer to begin write, given in 32bit words
Long	<i>nWords</i>	number of 32-bit words to write
Float FAR*	<i>pData</i>	floating point array holding data to load to RPx memory

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

WriteTargetV

Description: WriteTargetV is used to send data to a memory buffer located on a processor device.

WriteTargetV functions similarly to WriteTargetVEX but is designed for Legacy users. New users should refer to the WriteTargetVEX function.

Note: The Variant data array vData must be cast as a single for use with Matlab.

Prototype: `Function WriteTargetV(Target As String, nOS As Long, (vData As Variant) As Single) As Long`

Arguments:

String	<i>Target</i>	name of parameter tag.
Long	<i>nOS</i>	number of points to offset in buffer before starting write
Variant	<i>vData</i>	data array with the samples (this array must be cast as singles)

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

Sample Code

Description: This code writes the matrix new_coefs to the target Acq1.Filter1Coef

```

MATLAB new_coefs = [0.206572 0.413144 0.206572 -0.369527
0.195816];
new_coefs = single(new_coefs);
coefs =
DA.WriteTargetV('Acq1.Filter1Coef',0,new_coefs)

```


WriteTargetVEX

Description: WriteTargetVEX is used to send data to a memory buffer located on a processor device.

WriteTargetVEX takes data of type Variant and loads it to a target. The target is a parameter tag in a circuit running on a device and follows the form: DeviceName.ParameterTag. The DeviceName is the name given to the device by OpenWorkbench. The DstType specifies the format for the stored data. WriteTargetVEX will accept any type of Variant data array.

WriteTargetVEX has been added to TDevAcc for use with programming languages that use dynamic data typing, such as Python, but is also compatible with Matlab.

Note: Unlike WriteTarget and WriteTargetV, WriteTargetVEX does not require the vData array to be cast as a single.

Prototype: `Function WriteTargetVEX(Target As String, nOS As long, DstType As String , vData As Variant) As Long`

Arguments:

String	<i>Target</i>	name of parameter tag.
Long	<i>nOS</i>	number of points to offset in buffer before starting write.
String	<i>DstType</i>	format for storing data.

Floating Point (32-bit)	Word (32-bit)	Integer (16-bit)	Byte (8-bit)
F32	I32	I16	I8

Variant	<i>vData</i>	Any data array
---------	--------------	----------------

Returns:

Long	fails (0), succeeds (1)
------	-------------------------

Sample Code

Description: This code writes the matrix new_coefs to the target Acq1.Filter1Coef.

```
MATLAB
DA = actxcontrol('TDevAcc.X')
DA.ConnectServer('Local')
new_coefs = [0.206572 0.413144 0.206572 -0.369527
0.195816];
DA.WriteTargetVEX('Acq1.Filter1Coef',0,'F32',new_coefs
)
```

ReadTarget

Description:	ReadTarget is used to read data from a target to a buffer on the PC. ReadTarget functions similarly to ReadTargetVEX but is designed for Legacy users. New users should refer to the ReadTargetVEX function.	
Tech Notes:	ReadTargetV is used with the following components that have a data buffer: RamBuffer, SerialBuffer, AverageBuffer, LongDelay, LongDynDelay, ShortDelay, ShortDynDelay, Biquad, IIR,FIR, HrtfFir.	
Prototype:	<code>Function ReadTarget(Target As String, nOS As Long, nWords As Long, pBuf As Single) As Long</code>	
Arguments:		
String	<i>Target</i>	name of parameter tag.
Long	<i>nOS</i>	number of points to offset in buffer before starting read.
Long	<i>nWords</i>	number of 32-bit words to read (samples).
Float	<i>pBUF</i>	pointer to the buffer for storing the data.
Returns:		
Long	fails (0), succeeds (1)	

ReadTargetV

Description:	ReadTargetV is used to read data from a target to a buffer on the PC. ReadTargetV functions similarly to ReadTargetVEX but is designed for Legacy users. New users should refer to the ReadTargetVEX function. Note: The Variant data array returned is of type Single (F32).	
Tech Notes:	ReadTagV is used with the following components that have a data buffer: RamBuffer, Serial Buffer, Average Buffer, LongDelay, LongDynDelay, ShortDelay, ShortDynDelay, Biquad, IIR,FIR, HrtfFir.	
Prototype:	<code>Function ReadTargetV(Target As String, nOS As Long, nWords As Long) As Single</code>	
Arguments:		
String	<i>Target</i>	name of parameter tag.
Long	<i>nOS</i>	number of points to offset in buffer before starting read.
Long	<i>i</i>	number of 32-bit words to read (samples).
Returns:		
Variant	fails (-1 or NULL), succeeds (Variant of type Single)	

ReadTargetVEX

Description: ReadTargetVEX is used to read data from a target to a buffer on the PC.

ReadTargetVEX reads data from a parameter tag (Target) in a circuit running on a device that follows the form: DeviceName.ParameterTag. The DeviceName is the name given to the device by OpenWorkbench. The user must specify the source type (Srctype) of the parameter tag's data (F32, I32, I16, or 8-bit Integer) and the number of 32 bit words to read. It then converts it to one of five data formats (Double, Floating Point, Word, Integer, or 8-bit Integer) and stores it on a PC buffer.

If the data being read is shuffled, nWords is equivalent to the number of Samples in the Serial Buffer. If the data is compressed, nWords is equal to the number of points saved.

For example, if the data is compressed two-folded, 500 points of 1000 samples of the Serial Buffer have been read; nWords should be set to 500. For I8 format, a compression of 4, the number of points read from the buffer is 250 and nWords should be set to 250.

Note: ReadTargetVEX has been added to TDevAcc for use with programming languages that use dynamic data typing, such as Python, but is also compatible with Matlab.

Prototype: `Function ReadTargetVEX(Target As String, nOS As long, nWords As long, SrcType As String, DstType As String) As Variant`

Arguments:

String	<i>Target</i>	name of parameter tag.				
Long	<i>nOS</i>	number of points to offset in buffer before starting read.				
Long	<i>nWords</i>	number of 32-bit words to read (samples).				
String	<i>Srctype</i>	storage format type of data being read. Below is a list of the storage types.				
		Floating Point (32-bit)	Word (32-bit)	Integer (16-bit)	Byte (8-bit)	
		F32	I32	I16	I8	
String	<i>DstType</i>	format for storing data.				
		Double (64-bit float)	Floating Point (32-bit)	Word (32-bit)	Integer (16-bit)	Byte (8-bit)
		F64	F32	I32	I16	I8

Returns:

Variant	<i>-1 or empty</i>	not successful
Variant	<i>Array</i>	successful

Sample Code

Description: This **Sample Code** reads 5 32-bit words of data in single format from the parameter tag Acq1.Filter1Coef and returns it in double format.

MATLAB `DA = actxcontrol('TDevAcc.X')
DA.ConnectServer('Local')
coefs = DA.ReadTargetVEX('Acq1.Filter1Coef',0, 5,
'F32', 'F64')`

Hardware Information Retrieval

TDevAcc X

GetDeviceRCO

Description: GetDeviceRCO returns the full path name of the RCO file loaded to the specified device. A null string is returned if the device name is invalid or if no RCO file is loaded.

Prototype: `Function GetDeviceRCO(DeviceName As String) As String`

Arguments:

String	<i>DeviceName</i>	name of the device
		The device name is the name given to the device on the corresponding OpenWorkbench property sheet, for example, Amp1.

Returns:

String	<i>RCO file name</i>	returns the RCO file name and full path
--------	----------------------	---

See [tech note](#) for GetDeviceRCO.

GetDeviceSF

Description: GetDeviceSF (get device sample frequency) returns the exact sampling frequency of the RPx device. The target for GetDeviceSF is the hardware device. The device name is the name given to the device on the corresponding OpenWorkbench property sheet.

Prototype: `Function GetDeviceSF(Target As String) As float`

Arguments:

String	<i>Target</i>	name of the target (the OpenWorkbench Device name)
--------	---------------	--

Returns:

float		sampling frequency of the hardware device
-------	--	---

GetDeviceStatus

Description: GetDeviceStatus (get device status) returns the status of the device. The first three bits of the status are used by all RPx devices to indicate the following: a connection to the PC, a loaded RCO file, and a running circuit.

The target for GetDeviceStatus is the name of the hardware device. The device name is the name given to the device on the corresponding OpenWorkbench property sheet, for example, Amp1.

Prototype: `Function GetDeviceStatus(Target As String) As Long`

Arguments:

String	<i>Target</i>	name of the target (OpenWorkbench Device name)
--------	---------------	--

Returns:

Long	<i>xxxx0000000000</i>	connection status, first four bits check the status of the device
------	-----------------------	---

A bit-code value is set based on the status of the device.

Tech Notes: (all devices)

0 = false (low or none), 1 = true (high)

<i>Bit</i>	<i>Value (Enabled)</i>	<i>Status</i>
0	1	connected
1	2	cof loaded
2	4	circuit running
3	8	battery status (RL2, RA16)

Using the GetDeviceStatus simplifies the error checking routines (see below).

For best results, use bit-wise operations (0/1). Bits remain constant. Long values change as new bits are added to GetDeviceStatus().

If a circuit has previously been loaded to the RPx it will run when LoadCof fails, and the bit status of the device will read 0110(6) or 0111.

Tech Notes: (Medusa)

0 = false (low or none), 1 = true (high)

<i>Bit</i>	<i>Value (Enabled)</i>	<i>Status</i>
0	1	connected
1	2	cof loaded
2	4	circuit running
3	8	battery status
4	16	clipping on one or more channels
5	32	clipping occurred since last GetDeviceStatus

The Medusa has additional status values. Bit 4 indicates clipping is occurring on one or more channels. Bit 5 indicates that clipping has occurred since the last time GetDeviceStatus was called. Once GetDeviceStatus is called bit 5 is reset.

Note: When checking the status of the Medusa Base Station (RA16BA), ensure that a preamplifier is properly connected and turned on. Connection status (Bit 0) will always return a 0 when a preamplifier is not properly connected. Bit 5 (amplifier clipped since last call) is reset after GetStatus is called.

Tech Notes: (Barracuda)

0 = false (low or none), 1 = true (high)

Bit	Value (Enabled)	Status
0	1	connected
1	2	cof loaded
2	4	circuit running
3	8	n/a
4	16	not used
5	32	not used
6	64	system armed
7	128	circuit running
8	256	trigger enabled
9	512	auto clear DAC outs
10	1024	tick out
11	2048	clock out
12	4096	zTrigA
13	8192	zTrigB
14	16384	external trigger
15	32768	multiple trigger

GetDeviceType

Description: GetDeviceType returns the type of device as a long. The target is the name of the hardware device. The device name is the name given to the device by OpenWorkbench.

Prototype: `Function GetDeviceType(String Target) As Long`

Arguments:

String	<i>Target</i>	name of the target; for example, DeviceName
--------	---------------	---

Returns:

Long	a long that returns the DeviceType as an integer form (see below)
------	---

Tech Notes:	Device Type	Value
	RP2	0
	RL2	1
	RA16	2
	RV8	3

See [tech note](#) for GetDeviceType.

GetNextTag

Description: GetNextTag (get next parameter tag) returns the name of the parameter tag of a particular data type (such as integer or logical). The first call to the GetNextTag method must have a number other than 0 for DoFirst. All subsequent calls can pass the value 0 for DoFirst. The target for GetNextTag is the name of the hardware device. The DeviceName is the name given to the device by OpenWorkbench on the corresponding property sheet. Tags are indexed by alphabetical order.

Prototype: `Function GetNextTag(Target As String, ReqType As Long, DoFirst As Long) As String`

Arguments:

String	<i>Target</i>	name of target, form for target is DeviceName
Long	<i>ReqType</i>	data type associated with the target (see GetTargetType)
Long	<i>DoFirst</i>	The DoFirst parameter allows the user to specify whether to return the first tag (DoFirst = value > 0) or the next successive tag in the circuit (DoFirst = 0). If a zero is passed initially, this method will return a null string, therefore, a nonzero value must be passed initially.

Returns:

String	<i>parameter tag</i>	returns the next parameter tag of that data type in the sequence
--------	----------------------	--

Tech Notes:	Data Type	Value	Ascii Map
	Data (buffer)	68	"D"
	Integer	73	"I"
	Logical	78	"N"
	Coefficients	80	"P"
	Float	83	"S"

Sample Code

Description: Retrieves all the parameter tags associated with the buffer data type.

```
MATLAB
DA = actxcontrol('TDevAcc.X');
DA.ConnectServer('Local');
% Must specify a nonzero index for the first tag
target= DA.GetNextTag('Acq1',68,1)
for i=0:10 % Search for up to 10 tags
    target=DA.GetNextTag('Acq1',68,0);
    % Searching through tags until no more are found
    if(strcmp(target,''))
        break;
    else
        target %Output each tag found
    end
end
```

GetTargetType

Description: GetTargetType returns the data type of the specified target. The target is a parameter tag in a circuit running on a device and follows the form: DeviceName.ParameterTag. The DeviceName is the name given to the device by OpenWorkbench on the corresponding property sheet. GetTargetType returns 0 if the target is invalid.

Prototype: `Function GetTargetType(Target As String) As Long`

Arguments:

String *Target* name of target, in the form DeviceName.ParameterTag

Returns:

Long long that maps to an ASCII character (see Tech Notes below)

Tech Notes:	<i>Data Type</i>	<i>Value</i>	<i>Ascii Char</i>
	Data buffer / Delay line (DM)	68	"D"
	Integer	73	"I"
	Logical (1 or 0)	76	"L"
	Coefficient buffer (PM)	80	"P"
	Float (Single)	83	"S"
	Undefined (e.g. latch output)	65	"A"

GetTargetSize

Description: This function returns the size of a DM or PM buffer or scalar tag in 32-bit words. The tag is specified using the standard target naming convention and will return either the allocated size of the buffer or a one if the target is a scalar. A zero is returned for an invalid target.

Prototype: `Function GetTargetSize(Target As String) As Long`

Arguments:

String *Target* name of target, in the form DeviceName.ParameterTag

Returns:

Long 0 = error, 1 = scalar (Logic, Integer, or Float), >1 = size of the buffer

Examples

The example files below are installed with OpenEx Suite; however, they demonstrate some useful OpenDeveloper techniques. For some examples, two versions of the Matlab files are provided, one for use with the current version of Matlab and one for release 12 (designated by inclusion of "_R12" in the file name).

Files: OpenEx\Examples\TDevAccX_Example\VB\VBDemo1.exe,
Examples\TDevAccX_Example\VB\VBDemoWB.xpm

Hardware: RP2.1 Real-Time Processor

Overview: Uses TDevAcc methods to access the Workbench server and perform a frequency sweep based on the inputs presented by the user in the VBDemo1.exe Visual Basic form.

Note: In order to run this example correctly, you must open the Workbench file VBDemoWB.xpm prior to running VBDemo1.exe.

File: OpenEx\Examples\TTankX_Example\Matlab\TTankInterfaces Example\Main.m

Overview: Demonstrates the TTankInterfaces (ServerSelect, TankSelect, BlockSelect, and EventSelect). A button is included which returns the total number of events for the currently selected event.

Note: This example must be run using Matlab 7 or greater.

File: OpenEx\Examples\TTankX_Example\Matlab\Example1.m or Example1_R12.m

Overview: Demonstrates how data can be extracted from a tank and parsed.

File: OpenEx\Examples\TTankX_Example\Matlab\Example2.m or Example2_R12.m

Overview: Demonstrates how to extract filtered data from the tank. Data is filtered based on epoch events then events are extracted and parsed for later analysis and display.

File: OpenEx\Examples\TTankX_Example\Matlab\FilterArray.m or FilterArray_R12.m

Overview: Describes how to filter data from the OpenEx Tank.

Files: OpenEx\Examples\TTankX_Example\Matlab\InterSpikeInterval.m or InterSpikeInterval_R12.m

Overview: Demonstrates how to access tank data and parse events then plot the inter-spike intervals (ISI).

Files: OpenEx\Examples\TTankX_Example\Matlab\WaveReconstruction.m or
WaveReconstruction_R12.m

Overview: Demonstrates the steps used to reconstruct waveforms from events.

Known Anomalies

In OpenDeveloper, when using the GetDeviceType method in Matlab, VB, or VC++ the method returns a 0 regardless of the device. If possible, use OpenWorkbench in OpenEx to identify your hardware type.

In OpenDeveloper, when using the GetDeviceRCO method in Matlab, VB, or VC++ the method returns a null string regardless of the circuit loaded. If possible, use OpenWorkbench in OpenEx to load and run your circuit.

TtankX.GetEventCodes() causes a memory leak when running with Matlab 6.5.1. If invoked multiple times the system crashes. If possible, use Matlab 6.1 or lower until this issue is resolved.

ReadEvents/ReadEventsV may miss some events if there is a long interval between events in a block. Instead of trying to read all the events in the block at once (with start and stop time = 0), loop through and read the events in steps of 100 second (or less) intervals. Sample Matlab code:

```
ts = [ ];
% assumes there will never be more than 1000 events in an interval
maxevents = 1000;
% assumes a block will never be longer than 10000 seconds
maxtime = 10000;
% steps through block in 100 second intervals
steps = maxtime / 100;
for i = 1:steps
    % reads events in current 100 second interval
    events = ttank.ReadEventsV(maxevents, 'stor', 0, 0,
        ((i-1)*100), (i*100), 'ALL');
    if (events > 0)
        % if events were found, the timestamps are collected
        timestamps = ttank.ParseEvInfoV(0, events, 6);
        ts = cat(2, ts, timestamps(1,:));
    end
end
ts
```

Filtering methods, such as `SetFilterArray`, will always return 0 if a Store ID begins with any of the following characters: "-", "=", "(", ")", "<", ">", "!", a space or any number 0 to 9. When the TTank engine performs filtering of events in the Tank, the above characters will not be parsed correctly, and the store name will not be decoded properly.

The Global parameter `SortCode`, or argument `SortCode` for methods such as `ReadEvents` and `ReadEventsV`, cannot be used to define or condition the sort code of snippet events based on `SortId` results generated in `OpenSorter`. Even after `SetUseSortName` is called, the TTank server will use the default set of sort codes originally saved to the tank.

For example, the following Matlab code fails to cache Snip events with sort code 1, from the `OpenSorter` generated sort set "Sort1" and uses the default "TankSort" sort set instead.

```
SetSort = TT.SetUseSortName('Sort1')
SCode=1;
nEvents = TT.ReadEventsV(10000,'Snip',1,SCode,0.0,0.0,'All')
```

To use the alternate sort sets generated with `OpenSorter`, use the command `SetFilterWithDescEx` to set the sort code condition you want. Then use commands like `ReadEventsV` with the `SortCode` argument as 0, and the `Options` argument as 'FILTERED'. The Matlab code below will work to read Snip events of channel = 1 and sort code = 1 from the sort set saved as "Sort1".

```
SetSort1 = TT.SetUseSortName('Sort1');
TT.SetFilterWithDescEx('sort=1');
AllSort1 = TT.ReadEventsV(10000,'Snip',1,0,0.0,0.0,'FILTERED')
```

The most recent anomalies updates are available on the Web at www.tdt.com/T2Support/FlashHelp/System3TechNotes.htm.

Index

A

ActEventChange.....	92
ActiveBlock	80
ActiveEvent.....	86
ActiveServer	75
ActiveTank	77
AddServer	75
AllowActive	86
AllowEdit	75
AllowPopUp	78, 81

B

BlockChanged	85
BlockSelect	73, 80, 92

C

CheckServerConnection.....	98
CheckTank.....	29
ClearEpocIndexing	42
CloseConnection	100
CloseTank.....	5, 29
CodeToString.....	66
ConnectServer.....	5, 28, 98
CreateEpocIndexing	12, 42

D

DeleteServer	76
DFromToString	66

E

EventSelect.....	73, 86, 92
------------------	------------

G

GetCodeSpecs.....	60
GetDeviceRCO	106
GetDeviceSF.....	106
GetDeviceStatus.....	106
GetDeviceType	108
GetEnumServer	60
GetEnumTank.....	61
GetEpocCode	42
GetEpocsExV.....	44
GetEpocsV	43
GetError	61
GetEventCodes.....	62, 113
GetEvName	92
GetFilterTolerance	45
GetGlobalB	63
GetGlobalStringB	62
GetGlobalStringV	62

GetGlobalV	63
GetHotBlock.....	63
GetNextTag	109
GetNPer	67
GetSortName	30
GetStatus.....	64
GetSysMode	99
GetTargetSize.....	110
GetTargetType.....	110
GetTargetVal	101
GetValidTimeRangesV	15, 46
Global Parameters.....	6
H	
HideDetails	79, 81, 87
O	
OpenTank	5, 28
P	
ParseEv	37
ParseEvInfoV.....	9, 12, 20, 38
ParseEvV	9, 12, 20, 40
Q	
QryEpocAt	46
QryEpocAtV	12, 47
QueryBlockName.....	65

R

ReadEvents	31, 113
ReadEventsSimple	20, 25, 33
ReadEventsV	9, 12, 33, 113
ReadTarget	104
ReadTargetV.....	104
ReadWavesOnTimeRangeB	35
ReadWavesOnTimeRangeV	15, 34, 35
ReadWavesV	15, 36
Refresh	76, 79, 85, 91
ReleaseServer	5, 28
ResetFilters.....	12, 15, 48
ResetGlobals	67

S

SelectBlock	5, 30
ServerChanged.....	77
ServerSelect	73, 75, 92
SetEpocTimeFilter	48
SetEpocTimeFilterB	50
SetEpocTimeFilterV	49, 50
SetFilter	51
SetFilterArray	20, 52
SetFilterTolerance.....	55
SetFilterWithDesc	12, 53
SetFilterWithDescEx.....	15, 20, 54

SetGlobalB.....	68	ShowSize	89
SetGlobals	20, 69	ShowStart	83
SetGlobalsB.....	69	ShowStop.....	83
SetGlobalStringB	70	ShowTank	84, 89
SetGlobalStringV	15, 25, 70	ShowType	90
SetGlobalV.....	15, 25, 68	SingleClickSelect	78, 84, 90
SetRefEpoc.....	55	SortID.....	30, 40
SetRefEpocB	57	StringToEvCode.....	70
SetRefEpocV	56, 57	T	
SetSysMode	99	TankChanged	80
SetTargetVal.....	101	TankSelect.....	73, 77, 92
SetUseSortName.....	40	TDevAcc	97, 98
ShowBlock.....	87	TTankInterfaces	73, 92
ShowDataFormat.....	87	TTankX	5
ShowDate	81	U	
ShowDuration	82	UseBlock.....	90
ShowFirstTime.....	88	UseServer	79, 84, 91
ShowMemo.....	82	UseTank.....	85, 91
ShowOwner	83	W	
ShowSampleFrequency	88	WriteTarget	102
ShowServer	78, 89	WriteTargetV.....	102